

DOCUMENT DE RÉVISION

SmartAutoTrack

Structure du projet expliquée, dossier par dossier et fichier par fichier

Application web PHP de suivi automobile · Espaces admin, client, garage et technicien

Version du dépôt : 9f62d5b · 2 octobre 2026

Sommaire

1. Vue d'ensemble et architecture
2. config/ et includes/ : le socle commun
3. auth/, messages/, ajax/ et webhooks/
4. Les quatre espaces : admin/, client/, garage/, technicien/
5. assets/, uploads/, scripts/, sql/, tests/ et éléments hérités

1. Vue d'ensemble et architecture

1.1 Ce que fait l'application

SmartAutoTrack est une application web PHP de suivi de l'entretien automobile après-vente, pensée pour le Cameroun. Elle met en relation des **clients** (propriétaires de véhicules, particuliers ou entreprises), des **garages** et leurs **techniciens**, sous la supervision d'un **administrateur**.

Elle couvre : les véhicules, les demandes d'intervention, les anomalies, les réparations, le paiement Mobile Money d'une réparation terminée (CamPay), la messagerie interne, les notifications, un journal d'activité et un assistant IA (Hugging Face). Tout le code, l'interface et les messages sont en français.

1.2 Les quatre rôles

La base n'a **pas de colonne « rôle »**. Le rôle se déduit de la table dans laquelle l'identifiant de l'utilisateur apparaît : `administrateur`, `client`, `technicien`, ou `garage.idUtilisateur`. C'est la fonction `getUserRole()` de `config/roles.php` qui fait cette détection, une seule fois, à la connexion ; le rôle est ensuite gardé en session (`$_SESSION['role']`).

Rôle	Dossier	Création du compte	Ce qu'il peut faire
Client (particulier ou entreprise)	<code>client/</code>	Inscription libre (<code>auth/register.php</code>)	Gérer ses véhicules (son parc pour une entreprise), demander une intervention, suivre ses anomalies et réparations, payer une réparation terminée par Mobile Money, consulter son journal, son abonnement, le SAV, les messages et l'assistant IA.
Garage	<code>garage/</code>	Créé par un administrateur (<code>admin/garages.php</code>)	Traiter les demandes d'intervention (affecter un technicien, fixer date et priorité, ou annuler), gérer ses techniciens, enregistrer réparations et anomalies, consulter son historique et son journal.
Technicien	<code>technicien/</code>	Inscription (reste <code>EN_ATTENTE</code> jusqu'à validation) ou création par un garage / un admin (directement <code>VALIDE</code>)	Voir ses tâches, démarrer une intervention (<code>EN_COURS</code>), enregistrer la réparation (ce qui clôture l'intervention), signaler et suivre des anomalies, consulter son historique et son journal.
Administrateur	<code>admin/</code>	Aucune page ne le crée : lignes insérées en SQL, puis mot de passe via <code>php scripts/set_password.php</code>	Superviser clients, garages, techniciens, véhicules, interventions, réparations, anomalies ; valider, rejeter ou suspendre garages et techniciens ; consulter abonnements, transactions CamPay, statistiques, journal ; assistant IA sur les statistiques.

Un **technicien** ou un **garage** ne peut se connecter que si son statut vaut `VALIDE` (`isAccountUsable()`). Pour `EN_ATTENTE`, `REJETE` ou `SUSPENDU`, la page de connexion affiche un message explicite (`accountStatusMessage()`), et seulement *après* vérification du mot de passe : on ne révèle jamais le statut d'un compte à partir du seul email.

À retenir

- 4 rôles = 4 dossiers (`admin/`, `client/`, `garage/`, `technicien/`).
- Le rôle vient de la **table** où se trouve l'utilisateur, pas d'une colonne ni d'un paramètre envoyé par le navigateur.
- Garage et technicien doivent être `VALIDE` pour se connecter.

1.3 La stack technique

Couche	Technologie	Pourquoi / remarque
Serveur	Apache (XAMPP), modules <code>mod_headers</code> et <code>mod_authz_core</code>	Appliquent les fichiers <code>.htaccess</code> (en-têtes de sécurité, dossiers interdits).
Langage	PHP 8.2 (seule version vérifiée ; PHP 7.4 minimum à cause des fonctions fléchées <code>fn</code>)	PHP « classique », sans framework : chaque page est un script.

Couche	Technologie	Pourquoi / remarque
Base de données	MySQL 8 / MariaDB, base <code>charles</code> , accès par PDO	Requêtes préparées partout (protection contre l'injection SQL).
Front	HTML/CSS, jQuery 3.7.1, JavaScript maison (<code>assets/js/main.js</code> , <code>themes.js</code> , <code>assistant.js</code>), Font Awesome, Google Fonts	Aucune étape de compilation : les fichiers sont servis tels quels.
Services externes	CamPay (Mobile Money), Hugging Face (assistant IA), via cURL (<code>includes/http_client.php</code>)	Clés dans <code>config/local.php</code> ou des variables d'environnement, jamais dans le code.
Tests	Composer, PHPUnit 9.6, Guzzle 7.9 (en <code>require-dev</code>)	Composer ne sert qu'aux tests : l'application elle-même n'a aucune dépendance.

1.4 Architecture : « une page = un script PHP »

Il n'y a ni routeur, ni contrôleurs, ni framework. Chaque URL correspond à un fichier PHP (par exemple `technicien/reparations.php`) qui fait tout, **dans cet ordre** :

- Inclusions** : `config/config.php`, `config/database.php`, `config/roles.php` et, dans un espace, `includes/helpers.php` du rôle.
- Contrôle d'accès** : `requireRole('technicien')` (ou `requireAuth()` pour une page commune). Un visiteur non connecté part vers `auth/login.php` ; un utilisateur d'un autre rôle est renvoyé vers `dashboard.php`.
- Connexion à la base** : `$conn = (new Database())->getConnection();`
- Traitement du formulaire** (si `POST`) : vérification du jeton CSRF (`verifyCSRFToken()`), nettoyage et validation serveur des champs, contrôle que l'objet appartient bien à l'utilisateur, puis écriture (en transaction quand plusieurs tables changent ensemble).
- Requêtes de lecture** pour préparer les données à afficher.
- Affichage** : la page fixe `$pageTitle`, `$bodyClass` ... puis fait `include '../includes/header.php'`, écrit son HTML, et termine par `include '../includes/footer.php'`.

Pourquoi `header.php` est inclus après la logique ? Parce qu'une redirection (`header('Location: ...')` dans `redirect()`) n'est possible que tant qu'aucun HTML n'a été envoyé au navigateur. En traitant d'abord le `POST`, la page peut rediriger ou afficher des erreurs avant de produire le moindre octet de HTML.

Les appels JavaScript (AJAX) suivent le même principe dans `ajax/`, mais répondent en JSON : ils utilisent `requireJsonAuth()` (réponse 401/403 au lieu d'une redirection) et `verifyRequestCSRF()` (jeton accepté en champ `POST` ou en en-tête `X-CSRF-Token`).

1.5 Le cycle d'une requête

Que se passe-t-il quand le navigateur demande `technicien/reparations.php` ?

- Apache** applique `.htaccess` (en-têtes de sécurité, pas de cache pour les `.php`) puis exécute le script.
- `config/config.php`** (toujours inclus en premier) :
 - masque les erreurs PHP au visiteur (elles vont dans les logs ; `HCH_DEBUG=1` pour les voir en local) ;
 - calcule `BASE_PATH` et `SITE_URL` à partir de l'emplacement réel du projet (marche sous `/HCH/` comme à la racine d'un vhost) ;
 - définit les constantes de rôles (`ROLE_ADMIN`, `ROLE_CLIENT` ...) ;
 - démarré la **session** avec un cookie dédié `HCHSESSID` (`httponly`, `SameSite=Lax`, chemin limité à `BASE_PATH`) pour ne pas partager la session avec d'autres projets XAMPP ;
 - fournit les fonctions transverses : `requireAuth()`, `requireRole()`, `CSRF`, `h()` (échappement anti-XSS), `sanitize()`, validations, hachage `bcrypt` des mots de passe ; charge `includes/password_policy.php` et `includes/activity_log.php` ;
 - exécute `enforceActiveSession()` à chaque requête.

- 3 `enforceActiveSession()` : si quelqu'un est connecté, recharge son profil en base via `getUserProfile()` (`roles.php`). Si le compte a disparu, a changé de rôle, ou n'est plus utilisable (suspendu, rejeté), la session est vidée et reçoit un nouvel identifiant. *Pourquoi ?* Pour qu'une suspension par l'admin prenne effet **immédiatement**, pas seulement à la prochaine connexion.
- 4 `config/database.php` : la classe `Database` lit les paramètres (variables d'environnement `HCH_DB_*`, sinon `config/local.php`, sinon défauts XAMPP : `localhost`, `root` sans mot de passe) et ouvre une connexion **PDO** en `utf8mb4`, avec exceptions, résultats en tableaux associatifs et vraies requêtes préparées (`ATTR_EMULATE_PREPARES = false`). En cas d'échec sur le web : réponse 503 sans détail technique.
- 5 `config/roles.php` : `getUserRole()`, `getUserProfile()` (colonnes de `utilisateur` + colonnes du rôle), `findUserByEmail()`, `isAccountUsable()`, `accountStatusMessage()`. Lecture seule.
- 6 La page fait son **contrôle d'accès**, son **traitement POST** et ses **requêtes** (voir 1.4).
- 7 `includes/header.php` écrit le début du HTML : CSS (`style.css`, `themes.css`, feuilles propres à la page), jQuery, `main.js`, `themes.js`, et publie le jeton CSRF dans `<meta name="csrf-token">`. `main.js` relit ce jeton et l'ajoute en en-tête `X-CSRF-Token` à chaque requête AJAX jQuery.
- 8 Le HTML de la page, puis `includes/footer.php`, qui ferme la page.

À retenir

- `config.php` = session + outils communs + `enforceActiveSession()`, à chaque requête.
- Ordre d'une page : accès → POST (CSRF + validation) → lectures → `header.php` → HTML → `footer.php`.
- Un seul jeton CSRF par session, dans les formulaires (`csrf_token`) et en en-tête AJAX.

1.6 Arborescence racine commentée

Les éléments marqués **[HÉRITÉ]** sont d'anciennes versions du projet : ils sont encore dans le dossier mais **l'application actuelle ne les utilise pas**.

```

HCH/
├─ index.php                Vitrine publique (connecté → dashboard.php)
├─ dashboard.php           Aiguillage vers le tableau de bord du rôle
├─ profile.php             « Mon profil », commun aux 4 rôles
├─ settings.php           Aiguillage vers la page de paramètres du rôle
├─ .htaccess               En-têtes de sécurité, cache, fichiers interdits
├─ .gitignore              Fichiers exclus de Git (local.php, vendor/, tmp_*.txt...)
├─ README.md               Documentation du projet
├─
├─ admin/                  Espace administrateur (+ includes/helpers.php, sidebar.php)
├─ client/                 Espace client (+ includes/)
├─ garage/                 Espace garage (+ includes/)
├─ technicien/             Espace technicien (+ includes/)
├─ auth/                   login.php, register.php, logout.php
├─ messages/              Messagerie interne (index.php)
├─ ajax/                   Points d'accès JSON (session + CSRF) : CamPay, assistant IA,
                           notifications, messages, thème, détails, téléchargements
├─ webhooks/              campay.php : notification CamPay (sans session, signature vérifiée)
├─ config/                 config.php, database.php, roles.php, local.example.php,
                           local.php (secrets, non versionné) ; .htaccess : accès web interdit
├─ includes/              header.php, footer.php, payments.php, campay.php, ai.php,
                           http_client.php, login_throttle.php, activity_log.php,
                           password_policy.php
├─ assets/                 css/ (style, themes, *_v2 par rôle, home, auth_v2, assistant),
                           js/ (main.js, themes.js, assistant.js), img/ (logos)
├─ uploads/                Documents envoyés (techniciens/) ; accès web direct interdit
├─ scripts/                Scripts en ligne de commande (migrations, set_password.php) ;
                           accès web interdit
├─ sql/                    SCHEMA_ACTUEL.md + anciens schémas (init.sql, schema.sql,
                           seed_demo.sql, SCHEMA_NOTES.md : obsolètes)
├─ tests/                  PHPUnit : Unit/, Integration/, Functional/, bootstrap.php
├─ composer.json            Dépendances de test et commandes « composer test »
├─ composer.lock            Versions exactes installées
├─ phpunit.xml              Configuration des 3 suites de tests
├─ vendor/                 Dépendances Composer (PHPUnit, Guzzle) ; non versionné
├─
├─ composer.phar            Exécutable Composer (outil, pas du code applicatif)
├─ composer-setup.php       Installateur officiel de Composer (outil)
├─ .vscode/                 Réglages de l'éditeur VS Code
├─
├─ api/                    [HÉRITÉ] Ancienne API REST, autre base « smartautotrack »
├─ accueil/                [HÉRITÉ] Maquette HTML/JS statique de l'accueil
├─ admin-dashboard/        [HÉRITÉ] Maquette statique du tableau de bord admin
├─ Dashboard Technicien/   [HÉRITÉ] Maquette statique du tableau de bord technicien
├─ dashboard client/       [HÉRITÉ] Maquette statique du tableau de bord client
├─ charles (1).sql         [HÉRITÉ] Dump de l'ancien schéma (source de migrate_legacy_data.php)
├─ db_init.sql              [HÉRITÉ] Ancien script SQL, obsolète
├─ update_database.sql      [HÉRITÉ] Ancien script SQL, obsolète
├─ update_reparations_table.sql [HÉRITÉ] Ancien script SQL, obsolète
├─ fix_database_simple.sql  [HÉRITÉ] Ancien script SQL, obsolète
├─ tmp_cookies.txt, tmp_jar*.txt [HÉRITÉ] Cookies de tests manuels (ignorés par Git)

```

Attention : aucun des fichiers `.sql` ne sert à installer l'application actuelle : ils créent un ancien schéma (tables `users`, `vehicles` ...) qui ne correspond plus au code.

1.7 Les fichiers de la racine

Fichier	Rôle	Accès	Tables / dépendances
<code>index.php</code>	Page d'accueil publique (vitrine). Un utilisateur déjà connecté est renvoyé vers <code>dashboard.php</code> .	Public	Aucune lecture en base, aucun POST ; <code>includes/header.php</code> , <code>assets/css/home.css</code>
<code>dashboard.php</code>	Aiguillage : redirige vers <code>admin/</code> , <code>client/</code> , <code>garage/</code> ou <code>technicien/dashboard.php</code> selon <code>\$_SESSION['role']</code> .	Connecté (<code>requireAuth()</code>)	<code>config/config.php</code> uniquement
<code>profile.php</code>	« Mon profil » : infos personnelles, mot de passe (ancien exigé), et selon le rôle : entreprise, garage ou fiche technicien. Ne modifie que la ligne de <code>\$_SESSION['user_id']</code> .	Connecté, 4 rôles ; POST avec CSRF	<code>utilisateur</code> , <code>entreprise</code> , <code>garage</code> , <code>technicien</code> (écriture) ; <code>intervention</code> (lecture) ; <code>roles.php</code> , <code>password_policy.php</code>

Fichier	Rôle	Accès	Tables / dépendances
<code>settings.php</code>	Aiguillage du lien « Paramètres » vers la page du rôle (<code>admin/settings.php</code> , <code>client/settings.php</code> , <code>garage/parametres.php</code> , <code>technicien/parametres.php</code>).	Connecté (sinon connexion)	<code>config/config.php</code>
<code>.htaccess</code>	Page par défaut <code>index.php</code> ; en-têtes <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> ; <code>.php</code> jamais mis en cache ; interdit au web les <code>.sql</code> , <code>.md</code> , <code>.lock</code> , <code>.phar</code> , <code>composer.json</code> , <code>composer-setup.php</code> , <code>phpunit.xml</code> , <code>.gitignore</code> , <code>tmp_*.txt</code> .	Appliqué par Apache	<code>mod_headers</code> , <code>mod_authz_core</code>
<code>composer.json</code>	Déclare PHPUnit 9.6 et Guzzle 7.9 (en <code>require-dev</code>) et les commandes <code>composer test</code> , <code>test:unit</code> , <code>test:integration</code> , <code>test:functional</code> .	Ligne de commande	<code>vendor/</code> , <code>composer.lock</code>
<code>phpunit.xml</code>	Configure les 3 suites (<code>tests/Unit</code> , <code>tests/Integration</code> , <code>tests/Functional</code>), le fichier d'amorçage <code>tests/bootstrap.php</code> et l'URL testée <code>HCH_BASE_URL = http://localhost/HCH/</code> .	Ligne de commande	<code>tests/</code>

Pourquoi `dashboard.php` et `settings.php` existent ? Les liens communs (menu, connexion) n'ont pas à connaître le rôle : ils pointent vers une seule URL, et c'est le serveur qui aiguille selon le rôle enregistré en session — jamais selon une valeur envoyée par le navigateur.

À retenir

- Le code utile est dans les 4 espaces + `auth/`, `messages/`, `ajax/`, `webhooks/`, `config/`, `includes/`, `assets/`.
- `api/`, les maquettes statiques et les `.sql` de la racine sont hérités et inutilisés.
- `config/`, `scripts/` et `uploads/` sont interdits d'accès direct par le web (`.htaccess`).

Questions de révision

- Comment l'application connaît-elle le rôle d'un utilisateur, alors qu'il n'y a pas de colonne « rôle » ?**
`getUserRole()` (`config/roles.php`) cherche l'identifiant dans les tables `administrateur`, `client`, `technicien` et `garage.idUtilisateur`. Le rôle trouvé est stocké en session à la connexion.
- Décrivez l'ordre des étapes dans une page PHP de l'application.**
Inclusions (`config.php` ...), contrôle d'accès (`requireRole()`), connexion PDO, traitement POST (CSRF, validation, écriture), requêtes de lecture, puis `header.php`, HTML et `footer.php`.
- Pourquoi `header.php` est-il inclus après le traitement du formulaire ?**
Pour pouvoir encore rediriger (`header('Location: ...')`) : c'est impossible une fois que du HTML a été envoyé.
- À quoi sert `enforceActiveSession()` ?**
À chaque requête, elle vérifie en base que le compte existe, a toujours le même rôle et est utilisable ; sinon elle vide la session. Une suspension prend donc effet immédiatement.
- Quels dossiers de la racine ne sont pas utilisés par l'application actuelle ?**
`api/` (ancienne API, autre base `smartautotrack`), `accueil/`, `admin-dashboard/`, `Dashboad Technicien/`, `dashbord client/` (maquettes statiques), ainsi que les fichiers `.sql` et `tmp_*.txt` de la racine.

2. config/ et includes/ : le socle commun

Les pages de SmartAutoTrack (admin, garage, technicien, client, ajax...) ne réécrivent pas elles-mêmes la sécurité, la connexion à la base ou la validation des formulaires. Elles **incluent** un petit nombre de fichiers communs, rangés dans deux dossiers :

- `config/` : ce qui est chargé à **chaque requête** (constantes, session, contrôle d'accès, validations), la connexion à la base et la détection du rôle ;
- `includes/` : des briques réutilisables (gabarit HTML, journal d'activité, mots de passe, anti force brute, paiement CamPay, assistant IA, client HTTP).

Pourquoi ? Une règle écrite à un seul endroit s'applique partout de la même façon. Si on corrige `verifyCSRFToken()` ou la politique des mots de passe, toutes les pages en profitent sans être modifiées.

```
config/
├── .htaccess           Interdit tout accès web au dossier (Require all denied)
├── config.php          Chargé en premier partout : constantes, session, fonctions transverses
├── database.php        Classe Database : connexion PDO à MySQL
├── roles.php           Rôle et profil d'un utilisateur (getUserRole, getUserProfile...)
├── local.example.php   Modèle de configuration locale (versionné)
└── local.php           Votre configuration et vos secrets (NON versionné)

includes/
├── header.php          En-tête HTML commun (CSS, JS, jeton CSRF en <meta>)
├── footer.php          Pied de page commun (modales, toasts, compteurs de notifications)
├── password_policy.php Règles de robustesse des mots de passe
├── activity_log.php     Écriture et lecture cloisonnée du journal d'activité
├── login_throttle.php  Limitation des tentatives de connexion
├── http_client.php     Requête HTTP JSON sortante (cURL)
├── campay.php          Client de l'API de paiement Mobile Money CamPay
├── payments.php        Cycle d'un paiement de réparation (table paiement)
└── ai.php              Assistant IA (Hugging Face)
```

2.1 Les fichiers de config/

Fichier	Rôle	Accès	Tables / dépendances
<code>config/config.php</code>	Premier fichier inclus par toutes les pages. Calcule <code>BASE_PATH</code> et <code>SITE_URL</code> , définit les constantes, démarre la session, fournit les fonctions transverses (accès, CSRF, échappement, validations, hachage) et coupe la session d'un compte devenu inutilisable.	Inclus par toutes les pages (non appelé directement)	Charge <code>includes/password_policy.php</code> et <code>includes/activity_log.php</code> ; via <code>enforceActiveSession()</code> : <code>database.php</code> , <code>roles.php</code>
<code>config/database.php</code>	Classe <code>Database</code> : lit les identifiants puis ouvre une connexion PDO avec <code>getConnection()</code> .	Toute page qui lit la base	MySQL ; lit <code>local.php</code> et <code>HCH_DB_*</code>
<code>config/roles.php</code>	Détection du rôle (la base n'a pas de colonne « rôle ») et chargement du profil complet ; statut du compte.	Inclus explicitement là où nécessaire (connexion, pages, <code>enforceActiveSession()</code>)	Lecture seule : <code>utilisateur</code> , <code>administrateur</code> , <code>client</code> , <code>technicien</code> , <code>garage</code> , <code>particulier</code> , <code>entreprise</code>
<code>config/local.example.php</code>	Modèle à copier en <code>local.php</code> : renvoie un tableau de paramètres vides ou par défaut.	Versionné, jamais utilisé tel quel	—
<code>config/local.php</code>	Vos vrais paramètres (base, CamPay, Hugging Face).	Ignoré par Git (<code>.gitignore</code>)	Lu par <code>Database</code> et <code>appConfig()</code>
<code>config/.htaccess</code>	Refuse toute requête web vers le dossier <code>config/</code> .	Apache	—

Ce que fait config.php à chaque requête

- 1 **Erreurs** : `display_errors` vaut 0 (sauf si la variable d'environnement `HCH_DEBUG` vaut une valeur « vraie », par exemple 1) et les erreurs vont dans les logs. Le visiteur ne voit jamais un message SQL.
- 2 **Chemins** : `BASE_PATH` est calculé en comparant la racine du serveur web à l'emplacement réel du projet (défaut `/HCH/`). Le site fonctionne donc sous `http://localhost/HCH/` comme à la racine d'un vhost. `SITE_URL`, `UPLOAD_PATH` et `UPLOAD_URL` en découlent ; les dossiers `uploads/` et `uploads/techniciens/` sont créés s'ils manquent.
- 3 **Constantes** : rôles (`ROLE_ADMIN = 'admin'`, `ROLE_TECHNICIEN`, `ROLE_CLIENT`, `ROLE_GARAGE`), statuts (`STATUT_ACTIF`, `STATUT_PENDING`, `STATUT_REJETE` : héritées de l'ancien schéma, elles ne sont plus utilisées ailleurs dans le code, les statuts en base étant en majuscules comme `VALIDE` ou `EN_ATTENTE`) et types de notification (`NOTIF_ANOMALIE`, `NOTIF_INTERVENTION`...). Un garde `if (!defined('SITE_NAME'))` évite de les définir deux fois.
- 4 **Session** : cookie nommé `HCHSESSID`, limité au chemin `BASE_PATH`, `httponly`, `samesite=Lax`, `secure` en HTTPS. Pourquoi un nom dédié ? Pour ne pas partager la session avec les autres projets du même XAMPP. L'ancien cookie global `PHPSESSID` est supprimé (il provoquait des « connexions fantômes »).
- 5 **Fonctions** : définition de toutes les fonctions transverses (voir tableau 2.3), puis chargement de `password_policy.php` et `activity_log.php`.
- 6 **Contrôle du compte** : appel de `enforceActiveSession()` à la fin du fichier (voir 2.2).

2.2 Sécurité et accès : les fonctions clés

Contrôle d'accès

- `requireAuth()` : si `$_SESSION['user_id']` ou `$_SESSION['role']` manque, redirection vers `auth/login.php`.
- `requireRole($role)` : appelle `requireAuth()`, puis renvoie vers `dashboard.php` si le rôle ne correspond pas. `dashboard.php` aiguille ensuite l'utilisateur vers son propre espace.
- `requireJsonAuth($role = null)` : même contrôle pour les endpoints AJAX, mais la réponse est du JSON avec le code **401** (non connecté) ou **403** (mauvais rôle). Une redirection HTML n'aurait aucun sens pour un appel JavaScript.

Ces fonctions ne lisent **que la session**. Le rôle y est écrit une seule fois, à la connexion (`auth/login.php`), après avoir été déduit côté serveur par `getUserRole()`. Le navigateur ne peut donc jamais choisir son rôle.

Le rôle déduit des tables (config/roles.php)

- `getUserRole($conn, $id)` : une seule requête `UNION ALL ... LIMIT 1` cherche l'identifiant dans les quatre tables de rôle (`administrateur.idAdministrateur`, `client.idClient`, `technicien.idTechnicien`, `garage.idUtilisateur`) et renvoie `'admin'`, `'client'`, `'technicien'`, `'garage'` ou `null`. Un utilisateur n'est censé figurer que dans une seule de ces tables ; la requête n'a pas d'`ORDER BY`, elle ne garantit donc pas formellement un ordre de priorité.
- `getUserProfile($conn, $id)` : la ligne de `utilisateur` + le rôle + les colonnes propres au rôle (technicien : `idGarage`, `specialite`, `statutValidation`... ; client : `typeClient` puis `particulier` ou `entreprise` ; garage : `idGarage`, `nomGarage`, `statutGarage`...).
- `findUserByEmail()` : ligne brute de `utilisateur` (avec `motDePasse`), utilisée par la connexion.
- `isAccountUsable($profile)` : un technicien doit avoir `statutValidation = 'VALIDE'`, un garage `statutGarage = 'VALIDE'` ; clients et administrateurs sont toujours utilisables.
- `accountStatusMessage($profile)` : message clair selon le statut (`EN_ATTENTE`, `REJETE`, `SUSPENDU`). Il n'est affiché qu'**après** vérification du mot de passe, pour ne jamais révéler le statut d'un compte à partir du seul email.

enforceActiveSession() : couper l'accès tout de suite

Exécutée à chaque inclusion de `config.php`. Si un utilisateur est connecté, elle recharge son profil. Elle vide la session et régénère son identifiant si le compte n'existe plus, si son rôle a changé ou si `isAccountUsable()` est faux. **Pourquoi ?** Un technicien suspendu par l'administrateur perd l'accès **dès sa requête suivante**, sans attendre qu'il se déconnecte :

`requireAuth()` le renverra au login.

Protection CSRF

- `generateCSRFToken()` : crée au premier appel un jeton aléatoire (`bin2hex(random_bytes(32))`), gardé en session et réutilisé pendant toute la session. `includes/header.php` le publie dans `<meta name="csrf-token">`.

- `verifyCSRFToken($token)` : compare le jeton reçu à celui de la session avec `hash_equals()` (comparaison en temps constant, qui ne laisse pas deviner le jeton caractère par caractère).
- `verifyRequestCSRF()` : pour l'AJAX, accepte le champ POST `csrf_token` ou l'en-tête `X-CSRF-Token`. Cet en-tête est ajouté automatiquement par `assets/js/main.js` à toute requête jQuery vers le site (jamais vers un autre domaine, pour ne pas divulguer le jeton).

Pourquoi un jeton ? Un site malveillant peut faire envoyer un formulaire par le navigateur d'un utilisateur connecté (le cookie part tout seul), mais il ne connaît pas le jeton : la requête est refusée.

Saisies : nettoyer, valider, afficher

- `sanitize($data)` : `trim`, `strip_tags` puis `htmlspecialchars`. Les valeurs sont donc **stockées déjà encodées** (une apostrophe devient `'`).
- `h($value)` : échappement à l'affichage (anti-XSS) avec `double_encode = false`, pour ne pas ré-encoder ce que `sanitize()` a déjà encodé (on n'affiche pas `'`), tout en neutralisant les balises brutes.
- `validateLettersOnly()` et `validateModel()` commencent par `html_entity_decode()` : sans ce décodage, un nom valide comme « O'Brien », déjà encodé par `sanitize()`, serait refusé à tort.

Ces validations sont faites **côté serveur** : un contrôle dans le navigateur se contourne facilement, il n'est qu'un confort.

Fonction	Accepte	Exemple
<code>validateEmail()</code>	Format email valide (<code>FILTER_VALIDATE_EMAIL</code>)	<code>jean@exemple.cm</code>
<code>validateLettersOnly()</code>	Lettres (accents compris), espace, tiret, apostrophe ; ni chiffre ni symbole ; non vide	« Jean-Pierre », « O'Brien »
<code>validateDigitsOnly()</code>	Chiffres uniquement, non vide (téléphone)	<code>690000000</code>
<code>validatePlate()</code>	2 à 15 caractères, lettres et chiffres, espaces ou tirets au milieu seulement	« LT 123 AB », « CE-456-DF »
<code>normalizePlate()</code>	(transformation) majuscules, espaces multiples réduits à un seul	« lt 123 ab » → « LT 123 AB »
<code>validateModel()</code>	Commence par une lettre ou un chiffre ; lettres, chiffres, espace, <code>.</code> / <code>+</code> <code>!</code> <code>-</code> ; 50 caractères au plus	« RAV4 », « ID.4 », « C-HR »

À retenir

- `config.php` est inclus partout : session, constantes et fonctions transverses sont disponibles dans toutes les pages.
- Le rôle vient des tables (`getUserRole()`), jamais du navigateur ; il est mis en session à la connexion puis contrôlé par `requireRole()`.
- Chaîne des saisies : `sanitize()` à l'enregistrement, `validate...` côté serveur, `h()` à l'affichage.

2.3 Tableau des fonctions clés

Fonction	Fichier	Rôle
<code>appConfig(\$key, \$default)</code>	<code>config/config.php</code>	Paramètre : variable d'environnement, sinon <code>local.php</code> , sinon défaut
<code>appConfigBool()</code>	<code>config/config.php</code>	Valeur booléenne (<code>true/false</code> , <code>1/0</code> , <code>on/off</code> , <code>yes/no</code>)
<code>redirect(\$url)</code>	<code>config/config.php</code>	Redirection relative à <code>SITE_URL</code> puis <code>exit</code>
<code>requireAuth()</code>	<code>config/config.php</code>	Exige une connexion, sinon renvoie vers le login
<code>requireRole(\$role)</code>	<code>config/config.php</code>	Exige un rôle précis, sinon renvoie vers <code>dashboard.php</code>
<code>requireJsonAuth(\$role)</code>	<code>config/config.php</code>	Idem pour l'AJAX : réponse JSON 401 / 403

Fonction	Fichier	Rôle
<code>generateCSRFToken()</code>	<code>config/config.php</code>	Jeton CSRF unique par session
<code>verifyCSRFToken()</code> / <code>verifyRequestCSRF()</code>	<code>config/config.php</code>	Vérifie le jeton (champ POST ou en-tête X-CSRF-Token)
<code>sanitize()</code> / <code>h()</code>	<code>config/config.php</code>	Nettoyage à l'enregistrement / échappement à l'affichage
<code>validateEmail()</code> , <code>validateLettersOnly()</code> , <code>validateDigitsOnly()</code> , <code>validatePlate()</code> , <code>validateModel()</code>	<code>config/config.php</code>	Validations serveur des formulaires
<code>normalizePlate()</code>	<code>config/config.php</code>	Mise en forme d'une immatriculation
<code>formatDate()</code>	<code>config/config.php</code>	Date au format jj/mm/aaaa hh:mm
<code>hashPassword()</code> / <code>verifyPassword()</code>	<code>config/config.php</code>	<code>password_hash</code> (bcrypt par défaut) / <code>password_verify</code>
<code>destroySession()</code>	<code>config/config.php</code>	Déconnexion complète (session vidée, cookie expiré)
<code>enforceActiveSession()</code>	<code>config/config.php</code>	Coupe la session d'un compte supprimé, suspendu ou rejeté
<code>Database::getConnection()</code>	<code>config/database.php</code>	Connexion PDO (503 générique sur le web en cas d'échec)
<code>getUserRole()</code> / <code>getUserProfile()</code>	<code>config/roles.php</code>	Rôle déduit des tables / profil complet
<code>findUserByEmail()</code>	<code>config/roles.php</code>	Recherche d'un utilisateur à la connexion
<code>isAccountUsable()</code> / <code>accountStatusMessage()</code>	<code>config/roles.php</code>	Compte validé ? / message selon le statut
<code>passwordPolicyMissing()</code> / <code>passwordPolicyError()</code>	<code>includes/password_policy.php</code>	Règles non respectées / message d'erreur ou null
<code>log_activity()</code>	<code>includes/activity_log.php</code>	Seul point d'écriture de <code>journalactivites</code>
<code>activity_log_fetch()</code>	<code>includes/activity_log.php</code>	Seul point de lecture, cloisonné selon le rôle
<code>loginThrottleRetryAfter()</code> , <code>loginThrottleRecordFailure()</code> , <code>loginThrottleClear()</code>	<code>includes/login_throttle.php</code>	Anti force brute sur la connexion
<code>httpJsonRequest()</code>	<code>includes/http_client.php</code>	Appel HTTP JSON sortant (cURL)
<code>campayCollect()</code> , <code>campayTransactionStatus()</code> , <code>campayVerifyWebhookSignature()</code>	<code>includes/campay.php</code>	Demande de paiement, statut, vérification du webhook
<code>paymentStart()</code> , <code>paymentApplyCampayStatus()</code> , <code>paymentRefresh()</code>	<code>includes/payments.php</code>	Lancer un paiement, appliquer le statut CamPay, le rafraîchir
<code>aiChat()</code> , <code>aiClientContext()</code> , <code>aiAdminContext()</code> , <code>aiAllowMessage()</code>	<code>includes/ai.php</code>	Appel au modèle, contexte par rôle, limite de débit

2.4 Les fichiers de includes/

Fichier	Rôle	Accès	Tables / dépendances
<code>includes/header.php</code>	Début de page HTML : titre, CSS (<code>style.css</code> , <code>themes.css</code>), jQuery, <code>main.js</code> , <code>themes.js</code> , thème de la session, jeton CSRF en <code><meta></code> . Pour un utilisateur connecté, affiche la barre de navigation historique et ouvre <code><main></code> , sauf si <code>\$hideNavbar</code> est vrai (les pages « v2 » ont leur propre barre latérale). Variables facultatives posées par la page avant l'inclusion : <code>\$pageTitle</code> , <code>\$bodyClass</code> , <code>\$hideNavbar</code> , <code>\$extraStylesheets</code> , <code>\$extraFonts</code> .	Inclus par les pages après leur logique	<code>config.php</code> , <code>database.php</code> (s'ils ne sont pas déjà chargés)
<code>includes/footer.php</code>	Fin de page : referme <code><main></code> quand la barre de navigation a été affichée, modales, toasts, expose <code>SITE_URL</code> , <code>USER_ROLE</code> , <code>USER_ID</code> au JavaScript, rafraîchit les compteurs de notifications et de messages, marque les notifications comme lues.	Pendant de <code>header.php</code>	<code>ajax/notifications.php</code> , <code>ajax/message_count.php</code> , <code>ajax/mark_notification_read.php</code> , <code>ajax/mark_all_notifications_read.php</code>
<code>includes/password_policy.php</code>	Politique des mots de passe : au moins <code>PASSWORD_MIN_LENGTH</code> (8) caractères, une minuscule, une majuscule, un chiffre et un caractère spécial.	Chargé par <code>config.php</code> ; utilisé par <code>auth/register.php</code> , <code>profile.php</code> , <code>admin/garages.php</code> , <code>admin/techniciens.php</code> , <code>garage/techniciens.php</code> , <code>scripts/set_password.php</code>	Aucune table
<code>includes/activity_log.php</code>	Journal des actions métier importantes (pas les simples consultations).	Chargé par <code>config.php</code> (et par <code>payments.php</code>)	<code>journalactivites</code> (écriture/lecture) ; lecture de <code>intervention</code> (garage de l'intervention) et, pour l'affichage, jointures sur <code>utilisateur</code> , <code>administrateur</code> , <code>garage</code> , <code>technicien</code> , <code>client</code> , <code>intervention</code> , <code>vehicule</code> , <code>reparation</code> , <code>anomalie</code>
<code>includes/login_throttle.php</code>	Limite les échecs de connexion : 5 par email et 20 par adresse IP sur 15 minutes.	<code>auth/login.php</code>	Fichiers JSON dans <code>HCH_THROTTLE_DIR</code> ou le dossier temporaire du système

Fichier	Rôle	Accès	Tables / dépendances
<code>includes/http_client.php</code>	Requête HTTP JSON (cURL) ; lève une <code>RuntimeException</code> si le serveur est injoignable.	<code>campay.php</code> , <code>ai.php</code>	Extension cURL
<code>includes/campay.php</code>	Client de l'API CamPay (MTN / Orange, XAF) : URL démo ou production, jeton, normalisation du numéro (2376XXXXXXX), mode simulation, vérification du webhook (JWT HS256).	Inclus par <code>payments.php</code> ; ses fonctions sont aussi appelées par <code>webhooks/campay.php</code> (qui inclut <code>payments.php</code>)	<code>config.php</code> , <code>http_client.php</code>
<code>includes/payments.php</code>	Cycle d'un paiement de réparation : <code>EN_ATTENTE</code> → <code>PAYE</code> ou <code>ECHOUÉ</code> .	<code>client/reparations.php</code> , <code>ajax/campay_collect.php</code> , <code>ajax/campay_status.php</code> , <code>webhooks/campay.php</code> , <code>admin/transactions.php</code>	<code>paiement</code> , <code>reparation</code> , <code>intervention</code> , <code>notifications</code> , <code>journalactivites</code>
<code>includes/ai.php</code>	Assistant IA (Hugging Face, API compatible OpenAI) : contexte limité au périmètre de l'utilisateur, historique et limite de débit en session.	<code>client/assistant.php</code> , <code>admin/assistant.php</code> , <code>ajax/assistant_chat.php</code>	Lecture seule. Client : <code>vehicule</code> , <code>anomalie</code> , <code>intervention</code> , <code>reparation</code> . Admin : <code>client</code> , <code>vehicule</code> , <code>garage</code> , <code>technicien</code> , <code>intervention</code> , <code>anomalie</code> , <code>utilisateur</code> , <code>paiement</code> . Dépend de <code>config.php</code> , <code>http_client.php</code>

Journal d'activité : un seul point d'entrée

`log_activity()` enregistre qui, quoi, quand, sur quel élément et dans quel garage. Si l'appelant ne donne pas `idGarage` mais donne `idIntervention`, le garage est retrouvé automatiquement. `'idGarage' => false` force au contraire un garage vide (cas d'un garage recommandé qui n'est pas celui choisi par le client). `activity_log_fetch()` est la **seule** fonction qui lit le journal : l'admin voit tout, le garage uniquement son `idGarage`, le technicien ses propres actions (`idUtilisateur` ou `idTechnicien`), le client les événements de ses interventions (`i.idClient`). **Pourquoi centraliser ?** Une page mal écrite ne peut pas faire fuiter les activités d'un autre garage, puisque le filtre est construit au même endroit pour tout le monde.

Anti force brute (login_throttle.php)

Les compteurs sont stockés dans des **fichiers**, pas en session : un attaquant pourrait simplement ne pas renvoyer son cookie. Le nom de fichier est un hachage SHA-256 de la clé, donc l'email n'apparaît pas en clair sur le disque. L'email est mis en minuscules pour qu'un changement de casse ne contourne pas la limite. Une connexion réussie efface les échecs de l'email, pas ceux de l'IP. Quand la limite est atteinte, `auth/login.php` répond 429 avec l'en-tête `Retry-After`.

Paiement (campay.php + payments.php)

- 1 `paymentStart()` vérifie la réparation (terminée, coût > 0, appartenant au client) dans une transaction avec `FOR UPDATE` : deux clics simultanés ne créent pas deux paiements. Il enregistre ensuite un paiement `EN_ATTENTE` avec une référence `SAT-...`.
- 2 L'appel à `campayCollect()` se fait **hors transaction** (le réseau est lent) ; s'il échoue, le paiement passe `ECHOUÉ`.
- 3 Le webhook (`webhooks/campay.php`, signature vérifiée par `campayVerifyWebhookSignature()`) ou le suivi du statut (`paymentRefresh()`) appelle `paymentApplyCampayStatus()`.
- 4 Cette fonction est **idempotente** : un paiement `PAYE` n'est plus modifié et, grâce à `rowCount() === 1`, la notification « Paiement reçu » ne part qu'une fois. Un montant confirmé différent du montant attendu donne `ECHOUÉ` (vérification manuelle).

Un paiement `EN_ATTENTE` bloque une nouvelle tentative pendant `PAYMENT_PENDING_LOCK_SECONDS` (900 s). En démo, CamPay refuse plus de 25 XAF : `campayChargedAmount()` plafonne le montant débité à `CAMPAY_DEMO_MAX_AMOUNT`, mais le montant dû reste celui enregistré en base. `paymentsReady()` vérifie que la migration `scripts/migrate_structure.php` a ajouté les colonnes CamPay à `paiement`.

Assistant IA (ai.php)

Le modèle ne reçoit que des données du périmètre de l'utilisateur : pour un client, **ses** véhicules, anomalies non résolues, interventions et réparations (filtrés par `idClient`) ; pour un admin, uniquement des **totaux et agrégats**. Limites : 20 messages par tranche de 10 minutes (`AI_RATE_LIMIT`, `AI_RATE_WINDOW`), 1000 caractères par question, historique de 10 messages. `AIException` sépare le message affichable par tous du `$detail` technique, réservé à l'administrateur.

À retenir

- Le journal d'activité s'écrit et se lit par deux fonctions seulement, le cloisonnement par rôle est donc au même endroit.
- Paiement : transaction + `FOR UPDATE` contre les doublons, application du statut idempotente, contrôle du montant.
- Mot de passe : 8 caractères au minimum, minuscule, majuscule, chiffre, caractère spécial, contrôlés côté serveur par `passwordPolicyError()`.

2.5 Configuration locale : local.php et variables d'environnement

Aucun secret n'est écrit dans le code. Pour installer le projet, on copie `config/local.example.php` en `config/local.php` puis on le remplit. `local.php` est listé dans `.gitignore` : il n'est jamais envoyé sur GitHub, et `config/.htaccess` empêche de le télécharger depuis le web (ce fichier est pris en compte car le `httpd.conf` de XAMPP autorise les `.htaccess` dans `htdocs` avec `AllowOverride All`).

Ordre de lecture d'un paramètre avec `appConfig($key, $default)` :

- une variable d'environnement du même nom, si elle existe et n'est pas vide (posée par Apache avec `SetEnv` ou par le système) ;
- sinon la clé du tableau renvoyé par `config/local.php` (lu une seule fois par requête, grâce à un cache `static`) ;
- sinon la valeur par défaut passée à la fonction.

La base de données suit le même principe dans le constructeur de `Database`, mais avec d'autres noms :

Variable d'environnement	Clé dans local.php	Défaut
HCH_DB_HOST	db_host	localhost
HCH_DB_NAME	db_name	Charles
HCH_DB_USER	db_user	root
HCH_DB_PASS	db_pass	(vide)

Pour `HCH_DB_HOST`, `HCH_DB_NAME` et `HCH_DB_USER`, le code utilise `?:` : une variable vide est ignorée et on passe à `local.php`. Pour `HCH_DB_PASS`, le test est `!== false` : une variable définie à vide est respectée comme mot de passe vide (cas de `root` sans mot de passe sous XAMPP). La connexion PDO utilise `utf8mb4`, le mode exceptions, le fetch associatif et de vraies requêtes préparées (`ATTR_EMULATE_PREPARES => false`). En cas d'échec sur le web : réponse 503 « Service temporairement indisponible » et détail dans les logs ; en ligne de commande (tests, scripts) : `null`.

Paramètre	Utilisé par	Rôle
CAMPAY_USE_DEMO, CAMPAY_BASE_URL	campay.php	API de démo (défaut) ou de production, ou URL imposée
CAMPAY_TOKEN ou CAMPAY_USERNAME + CAMPAY_PASSWORD	campay.php	Authentification ; le jeton permanent est prioritaire
CAMPAY_WEBHOOK_KEY	campay.php	Clé de vérification de la signature du webhook
CAMPAY_SIMULATION	campay.php	Aucun appel réseau, chaque paiement réussit (développement)
CAMPAY_DEMO_MAX_AMOUNT	campay.php	Plafond débité en démo (défaut 25, 0 = aucun)

Paramètre	Utilisé par	Rôle
HF_TOKEN , HF_MODEL , HF_API_URL	ai.php	Jeton, modèle (défaut Qwen/Qwen2.5-7B-Instruct:fastest) et point d'accès Hugging Face ; sans jeton, l'assistant est indisponible
HCH_DEBUG	config.php (environnement seulement)	Affiche les erreurs PHP ; développement uniquement
HCH_THROTTLE_DIR	login_throttle.php (environnement seulement)	Dossier des compteurs de tentatives

CAMPAY_APP_ID figure dans local.example.php mais n'est lu nulle part dans le code.

À retenir

- Secrets dans local.php (non versionné) ou dans l'environnement, jamais dans le code.
- Priorité : variable d'environnement > local.php > valeur par défaut.

Questions de révision

- Pourquoi la table utilisateur n'a-t-elle pas de colonne « rôle », et comment le rôle est-il connu ?**
 getUserRole() cherche l'identifiant dans administrateur, client, technicien et garage.idUtilisateur. Le rôle est écrit en session à la connexion, puis contrôlé par requireRole() ; le navigateur ne le transmet jamais.
- Un administrateur suspend un technicien connecté. Que se passe-t-il ?**
 À sa requête suivante, enforceActiveSession() (exécutée par config.php) voit que isAccountUsable() est faux, vide la session et régénère son identifiant ; requireAuth() le renvoie alors au login.
- Quelle différence entre sanitize() et h() ?**
 sanitize() nettoie et encode une saisie avant de l'enregistrer ; h() échappe une valeur au moment de l'afficher, sans ré-encoder ce qui l'est déjà (double_encode = false).
- Comment fonctionne la protection CSRF pour les appels AJAX ?**
 Le jeton de session est publié dans <meta name="csrf-token"> par header.php ; main.js l'envoie dans l'en-tête X-CSRF-Token ; verifyRequestCSRF() le compare avec hash_equals().
- Où mettre le mot de passe MySQL et la clé CamPay, et pourquoi ?**
 Dans config/local.php (ignoré par Git, protégé par config/.htaccess) ou en variables d'environnement, qui sont prioritaires. Ainsi aucun secret n'est publié avec le code.

3. auth/, messages/, ajax/ et webhooks/

Ces quatre dossiers regroupent les points d'entrée « transversaux » de l'application, qui ne sont pas propres à un seul espace (client, technicien, garage, admin) :

- `auth/` : entrer dans l'application (connexion, inscription) et en sortir (déconnexion) ;
- `messages/` : la messagerie interne, commune aux quatre rôles ;
- `ajax/` : 21 petits scripts appelés en arrière-plan par le JavaScript des pages (ils renvoient du JSON ou un fichier, jamais une page complète) ;
- `webhooks/` : un script appelé non pas par un navigateur, mais par le serveur de CamPay (paiement Mobile Money).

```
auth/  
├─ login.php      connexion (tous rôles)  
├─ register.php   inscription (client ou technicien)  
└─ logout.php     déconnexion  
messages/  
└─ index.php      messagerie interne (4 rôles)  
ajax/  
21 endpoints JSON / téléchargement  
webhooks/  
└─ campay.php     notification de fin de paiement CamPay
```

3.1 Le dossier auth/

auth/login.php : la connexion

Page publique, commune à tous les rôles. Le formulaire ne contient que l'**email**, le **mot de passe** et un **jeton CSRF**. Point essentiel : **le rôle n'est jamais envoyé par le navigateur**. Il est déduit côté serveur par `getUserRole()` (`config/roles.php`), qui cherche l'identifiant de l'utilisateur dans les tables `administrateur`, `client`, `technicien` et `garage`. Pourquoi ? Parce que tout ce qui vient du navigateur peut être modifié : si le rôle venait du formulaire, un client pourrait se déclarer « admin ».

Limitation des tentatives (anti force brute) — `includes/login_throttle.php` :

- fenêtre glissante de **15 minutes** (`LOGIN_THROTTLE_WINDOW = 900`) ;
- au plus **5 échecs par email** et **20 échecs par adresse IP** dans cette fenêtre ;
- au-delà, la page répond avec le code HTTP **429** et un en-tête `Retry-After`, et affiche le nombre de minutes à attendre ;
- les compteurs sont stockés dans des **fichiers** (dossier temporaire du système ou `HCH_THROTTLE_DIR`), et non en session : un attaquant pourrait simplement ne pas renvoyer son cookie de session pour remettre son compteur à zéro. Le nom du fichier est un hachage SHA-256 de la clé, donc l'email n'apparaît pas en clair sur le disque ;
- l'email est mis en minuscules et sans espaces avant comptage, pour qu'une variante de casse ne contourne pas la limite.

Messages d'erreur : si l'email n'existe pas ou si le mot de passe est faux, le message est volontairement le même (« Email ou mot de passe incorrect. ») pour ne pas révéler quels emails sont inscrits. Si les identifiants sont corrects mais que le compte n'est pas utilisable (technicien ou garage `EN_ATTENTE`, `REJETE` ou `SUSPENDU`), `isAccountUsable()` renvoie faux et `accountStatusMessage()` affiche un message précis. Ce message n'est donné qu'**après** vérification du mot de passe, pour ne jamais révéler le statut d'un compte à partir du seul email.

Régénération de session : en cas de succès, `session_regenerate_id(true)` est appelé **avant** d'écrire dans `$_SESSION`. Cela protège contre la **fixation de session** : un attaquant qui aurait réussi à imposer un identifiant de session à la victime avant sa connexion ne peut pas le réutiliser, car cet identifiant est remplacé au moment où la victime se connecte.

La session reçoit ensuite `user_id`, `nom`, `prenom`, `email`, `role` et `statut` (ancienne convention `actif` / `pending` / `rejete`, gardée pour les pages qui la lisent encore). Le cookie de session s'appelle `HCHSESSID` et il est configuré dans `config/config.php` avec `httponly` et `samesite=Lax`.

Parcours détaillé : la connexion

- 1 L'utilisateur envoie le formulaire (email, mot de passe, `csrf_token`).
- 2 `verifyCSRFToken()` compare le jeton à celui de la session (en temps constant, avec `hash_equals`). S'il est invalide : « Token de sécurité invalide. »
- 3 Contrôle des champs vides.
- 4 `loginThrottleRetryAfter(email, IP)` : si la limite est atteinte, réponse 429 + `Retry-After` et arrêt.

- 5 `findUserByEmail()` lit `utilisateur`, puis `getUserProfile()` déduit le rôle et charge les colonnes propres au rôle.
- 6 `verifyPassword()` (`password_verify`) compare le mot de passe au hachage de la colonne `motDePasse`. En cas d'échec : `loginThrottleRecordFailure()` et message générique.
- 7 Si le compte n'est pas utilisable : le compteur de l'email est remis à zéro (`loginThrottleClear()`) et le message de statut est affiché (pas d'accès au tableau de bord).
- 8 Sinon : `session_regenerate_id(true)`, `loginThrottleClear()`, remplissage de `$_SESSION`.
- 9 Redirection selon le rôle réel : `admin/dashboard.php`, `technicien/dashboard.php`, `client/dashboard.php` OU `garage/dashboard.php`.

auth/register.php : l'inscription

Page publique qui ne permet de créer que deux types de comptes : **client** et **technicien**. Les comptes garage et administrateur ne se créent pas ici (le rôle reçu est refusé s'il n'est pas dans `['client', 'technicien']`). Ici, contrairement à la connexion, le rôle vient bien du formulaire, mais seules ces deux valeurs sont acceptées.

Cas	Champs demandés	Tables écrites	État du compte
Client particulier	champs communs + un premier véhicule (marque, modèle, immatriculation)	<code>utilisateur</code> , <code>client</code> (<code>typeClient = PARTICULIER</code>), <code>particulier</code> , <code>vehicule</code>	actif tout de suite
Client entreprise	champs communs + raison sociale (obligatoire), adresse (facultative) ; pas de véhicule, la flotte est ajoutée ensuite depuis le tableau de bord	<code>utilisateur</code> , <code>client</code> (<code>typeClient = ENTREPRISE</code>), <code>entreprise</code>	actif tout de suite
Technicien	champs communs + compétences, expérience, documents justificatifs	<code>utilisateur</code> , <code>technicien</code> (<code>statutValidation = EN_ATTENTE</code>), <code>technician_documents</code> , <code>notifications</code>	en attente de validation par un administrateur

Contrôles des champs communs : nom et prénom en lettres uniquement (`validateLettersOnly`), email valide (`validateEmail`), téléphone en chiffres uniquement (`validateDigitsOnly`), confirmation identique au mot de passe, email pas déjà présent dans `utilisateur`. Pour le véhicule : marque en lettres, modèle via `validateModel`, immatriculation normalisée par `normalizePlate` puis vérifiée par `validatePlate`.

Règles de mot de passe (`includes/password_policy.php`, chargé par `config/config.php`) : au moins **8 caractères** (`PASSWORD_MIN_LENGTH`), une **minuscule**, une **majuscule**, un **chiffre** et un **caractère spécial**. `passwordPolicyError()` construit le message listant ce qui manque. Le même fichier sert au profil, à la création de garage/technicien et au script `scripts/set_password.php`. Le contrôle dans le navigateur (`assets/js/main.js`, attribut `data-password-policy`) n'est qu'un confort : **c'est la validation serveur qui fait foi**, car le JavaScript peut être désactivé ou contourné. Le mot de passe est stocké haché par `hashPassword()` (`password_hash`, algorithme `PASSWORD_DEFAULT`).

Documents du technicien :

- types acceptés : PDF, JPG, PNG, **vérifiés sur le contenu réel** du fichier avec `finfo` (et non d'après l'extension, qu'on peut falsifier) ;
- taille maximale : **5 Mo** par fichier ;
- le fichier est renommé de façon aléatoire (`idUtilisateur_` + 32 caractères hexadécimaux issus de `random_bytes`) avec l'extension correspondant au type réel, dans `uploads/techniciens/` ;
- un type de document indicatif est déduit pour aider l'administrateur : `diplome` ou `certificat` si le nom d'origine contient ce mot, sinon `photo` si le fichier est une image (JPG/PNG), sinon `autre` ;
- un fichier refusé n'annule pas l'inscription : il est seulement signalé dans les erreurs ;
- le dossier `uploads/` est fermé à tout accès direct par `uploads/.htaccess` (`Require all denied`) : les fichiers ne sont servis que par `ajax/download_document.php`, après contrôle des droits.

Parcours détaillé : l'inscription

- 1 L'utilisateur choisit « Client » ou « Technicien », remplit le formulaire (`enctype="multipart/form-data"` pour les fichiers) et l'envoie.
- 2 Vérification du jeton CSRF.
- 3 Validation des champs communs, du mot de passe (`passwordPolicyError`) et du rôle.

- 4 Validation spécifique : véhicule pour un particulier, raison sociale pour une entreprise, compétences et expérience pour un technicien.
- 5 Vérification que l'email n'est pas déjà utilisé.
- 6 `beginTransaction()` : insertion dans `utilisateur` (mot de passe haché), puis dans les tables du rôle (`client` + particulier / entreprise + `vehicule`, OU `technicien` en `EN_ATTENTE`).
- 7 Pour un technicien : traitement des documents envoyés et insertion dans `technician_documents`, puis création d'une notification « Nouvelle demande de technicien » dans `notifications`, adressée à `user_id = 1` (valeur écrite en dur : on suppose que l'utilisateur n°1 est l'administrateur).
- 8 `commit()`. En cas d'exception, `rollback()` annule tout : il ne reste jamais un utilisateur « à moitié créé ».
- 9 Message de succès (différent pour client et technicien), puis redirection vers `login.php` au bout de 3 secondes.

auth/logout.php : la déconnexion

Appelle `destroySession()` (`config/config.php`), qui vide `$_SESSION`, fait expirer le cookie de session et détruit la session, puis redirige vers `auth/login.php?deconnecte=1`, où un message confirme la déconnexion.

À retenir

- À la connexion, le rôle est **déduit côté serveur**, jamais lu dans le formulaire.
- Trois protections à la connexion : jeton CSRF, limitation 5 échecs/email et 20 échecs/IP sur 15 min (HTTP 429), régénération de l'identifiant de session.
- L'inscription se fait en **une seule transaction** ; un technicien reste `EN_ATTENTE` jusqu'à la validation par un administrateur.

3.2 Le dossier messages/

`messages/index.php` est la **messagerie interne**, la même page pour les quatre rôles (seul l'habillage change : feuille de style `client_v2.css`, `garage_v2.css`, `technicien_v2.css` ou `admin_v2.css`). Accès : tout utilisateur connecté (`requireAuth()`).

- **Liste des conversations** : une seule requête regroupe les messages de `messages` par interlocuteur, avec la date du dernier message, le dernier contenu et le nombre de non lus. Toutes les requêtes sont limitées à `$_SESSION['user_id']` : chacun ne voit que ses propres conversations.
- **Choix d'un destinataire** : on ne peut écrire qu'à un utilisateur d'un **autre rôle** que le sien ; les techniciens et garages non validés (`statutValidation` / `statutGarage` différent de `VALIDE`) n'apparaissent pas.
- `GET contact=<id>` ouvre directement une conversation (bouton « Contacter » d'une réparation, par exemple).
- La page ne traite **aucun POST** : le chargement d'une conversation passe par `ajax/load_conversation.php` et l'envoi par `ajax/send_message.php`. Pourquoi ? Pour ne pas recharger toute la page à chaque message.

La règle « rôle différent » est appliquée **deux fois** : dans la liste de la page (confort) et dans `send_message.php` (sécurité), car quelqu'un pourrait envoyer une requête directement à l'endpoint sans passer par la page.

3.3 Le dossier ajax/

Un endpoint AJAX est un script appelé par le JavaScript de la page (jQuery `$.ajax`) pour obtenir ou modifier une donnée sans recharger la page. Les règles communes :

- réponse JSON (header('Content-Type: application/json')), sauf pour les téléchargements ;
- contrôle de connexion par `requireJsonAuth()` (réponse 401 si non connecté, 403 si le rôle exigé ne correspond pas) ou par un test direct de `$_SESSION['user_id']` (dans ce cas, certains fichiers — `message_count.php`, `notifications.php`, `load_conversation.php`, `send_message.php`, `get_vehicule.php` — répondent `success: false` avec un statut HTTP 200). On renvoie du JSON plutôt qu'une redirection, car le JavaScript ne saurait pas quoi faire d'une page de connexion. Exception : les trois téléchargements (`download_*.php`) utilisent `requireAuth()`, qui redirige vers la page de connexion ;
- les modifications se font en **POST avec jeton CSRF**, vérifié par `verifyRequestCSRF()` qui accepte le champ `csrf_token` ou l'en-tête `X-CSRF-Token` (ajouté automatiquement par `assets/js/main.js`). `send_message.php` utilise `verifyCSRFToken()` et n'accepte que le champ `csrf_token`. Seule exception : `load_conversation.php`, appelé en GET sans jeton, marque aussi les messages reçus comme lus (modification mineure, limitée aux messages destinés à l'utilisateur connecté) ;

- les lectures sont **cloisonnées par rôle** : chaque requête SQL contient une condition qui limite le résultat à ce que l'utilisateur a le droit de voir (ex. `v.idClient = ?` pour un client).

Fichier	Rôle	Accès	Tables / dépendances
<code>assistant_chat.php</code>	Assistant IA : envoie la question avec un contexte propre au rôle et l'historique récent ; <code>action=reset</code> efface l'historique. Longueur limitée à 1000 caractères (<code>AI_MAX_MESSAGE_LENGTH</code> , 400 sinon) et débit limité à 20 messages par 10 minutes et par session (<code>aiAllowMessage</code> , 429 si dépassé).	client ou admin, POST + CSRF	<code>includes/ai.php</code> ; appelé par <code>assets/js/assistant.js</code>
<code>campay_collect.php</code>	Lance le paiement Mobile Money d'une réparation ; renvoie <code>paiement_id</code> , montant, code USSD et opérateur.	client, POST + CSRF	<code>paiement</code> via <code>paymentStart()</code> (<code>includes/payments.php</code>) ; appelé par <code>client/reparations.php</code>
<code>campay_status.php</code>	Interrogé régulièrement pour connaître l'état d'un paiement en cours (404 si le paiement n'appartient pas au client).	client, POST + CSRF	<code>paiement</code> via <code>paymentFindForClient()</code> puis <code>paymentRefresh()</code> ; en cas d'erreur passagère, renvoie <code>EN_ATTENTE</code> pour que la page réessaie
<code>download_document.php</code>	Affiche ou télécharge (<code>&download=1</code>) un document justificatif de technicien. Chemin contrôlé avec <code>realpath</code> (doit être dans <code>uploads/techniciens/</code>), type MIME relu sur le contenu (415 sinon), en-têtes <code>nosniff</code> et CSP.	admin, ou le technicien propriétaire (404 sinon)	<code>technician_documents</code>
<code>download_garage_document.php</code>	Même principe pour les documents d'un garage (fichiers dans <code>uploads/garages/</code>) ; le nom de téléchargement est dérivé du type de document, car <code>documentgarage</code> ne garde pas le nom d'origine.	admin, ou le garage propriétaire (404 sinon)	<code>documentgarage</code> , <code>garage</code>
<code>download_report.php</code>	Télécharge le rapport d'une réparation sous forme de fichier HTML autonome imprimable.	connecté, cloisonné : client (ses véhicules), technicien (ses réparations), garage (ses interventions), admin (tout)	<code>reparation</code> , <code>intervention</code> , <code>vehicule</code> , <code>utilisateur</code> , <code>garage</code>
<code>get_anomaly_details.php</code>	Détail d'une anomalie avec son véhicule.	connecté, filtre par rôle	<code>anomalie</code> , <code>vehicule</code> , <code>intervention</code> , <code>garage</code>
<code>get_client_details.php</code>	Fiche d'un client et liste de ses véhicules (fenêtres de détail de l'administration).	admin	<code>utilisateur</code> , <code>client</code> , <code>vehicule</code>
<code>get_reparation_details.php</code>	Détail d'une réparation (véhicule, technicien).	connecté, filtre par rôle	<code>reparation</code> , <code>intervention</code> , <code>vehicule</code> , <code>utilisateur</code> , <code>garage</code>
<code>get_report_details.php</code>	Détail d'un rapport rédigé par le technicien (un « rapport » est une ligne de <code>reparation</code>).	technicien, ses propres réparations	<code>reparation</code> , <code>intervention</code> , <code>vehicule</code> , <code>utilisateur</code>
<code>get_task_details.php</code>	Détail d'une tâche (intervention assignée) avec client, véhicule et, si terminée, le dernier rapport.	technicien, ses propres interventions	<code>intervention</code> , <code>vehicule</code> , <code>utilisateur</code> , <code>reparation</code>

Fichier	Rôle	Accès	Tables / dépendances
<code>get_technician_details.php</code>	Fiche d'un technicien (compétences, expérience, statut de validation).	admin	<code>utilisateur</code> , <code>technicien</code>
<code>get_technician_documents.php</code>	Liste des documents d'un technicien (métadonnées seulement) pour la validation de l'inscription.	admin	<code>utilisateur</code> , <code>technicien</code> , <code>technician_documents</code>
<code>get_vehicule.php</code>	Données d'un véhicule (ex. préremplir le formulaire de modification de <code>client/vehicules.php</code>).	connecté, filtre par rôle	<code>vehicule</code> , <code>intervention</code> , <code>garage</code>
<code>load_conversation.php</code>	Charge la conversation avec un contact puis marque comme lus les messages reçus de lui.	connecté	<code>utilisateur</code> (lecture), <code>messages</code> (lecture, mise à jour de lu)
<code>mark_all_notifications_read.php</code>	Marque toutes ses notifications comme lues.	connecté, POST + CSRF	<code>notifications</code>
<code>mark_notification_read.php</code>	Marque une notification comme lue ; la condition <code>user_id</code> empêche de toucher celle d'un autre.	connecté, POST + CSRF	<code>notifications</code>
<code>message_count.php</code>	Nombre de messages non lus, pour le badge de la barre de navigation.	connecté	<code>messages</code> ; appelé par <code>assets/js/main.js</code> et <code>includes/footer.php</code>
<code>notifications.php</code>	Les 10 dernières notifications et le nombre de non lues, pour le menu déroulant.	connecté	<code>notifications</code> ; appelé par <code>assets/js/main.js</code> et <code>includes/footer.php</code>
<code>save_theme.php</code>	Enregistre le thème choisi (liste blanche : light, dark, blue, green, purple, orange), en base et en session.	connecté, POST + CSRF	<code>utilisateur</code> (<code>themePreference</code>) ; appelé par <code>assets/js/themes.js</code>
<code>send_message.php</code>	Envoie un message interne et crée la notification du destinataire. Le destinataire doit exister, avoir un compte utilisable et un rôle différent ; sinon message générique « Destinataire introuvable ».	connecté, POST + CSRF	<code>utilisateur</code> (lecture), <code>messages</code> , <code>notifications</code> (écriture)

À retenir

- Lecture = requête SQL cloisonnée par rôle (en général en GET) ; écriture = POST + jeton CSRF (seule exception : le marquage « lu » de `load_conversation.php`).
- Les fichiers envoyés ne sont jamais accessibles directement : ils passent par `download_document.php` / `download_garage_document.php`, qui vérifient les droits, le chemin et le type réel.

3.4 Le dossier webhooks/

`webhooks/campay.php` est l'adresse que CamPay appelle lorsqu'une transaction se termine (URL à déclarer dans le tableau de bord CamPay : `<SITE_URL>webhooks/campay.php`). C'est un serveur qui l'appelle, pas un utilisateur : il n'y a donc **ni session ni jeton CSRF**. La sécurité repose sur trois mécanismes :

- Signature JWT** : la notification doit porter un paramètre `signature`, un JWT signé en HS256 avec la clé secrète `CAMPAY_WEBHOOK_KEY` (si cette clé n'est pas configurée, tout est refusé). `campayVerifyWebhookSignature()` (`includes/campay.php`) vérifie l'algorithme, recalcule la signature HMAC-SHA256, la compare en temps constant (`hash_equals`) et refuse un jeton expiré (`exp`). Sinon : HTTP 403.
- Paiement existant** : `paymentFindByCampayReference()` doit retrouver un paiement de la table `paiement` par sa référence CamPay (ou sa référence externe). Sans référence : HTTP 400 ; référence inconnue : réponse JSON `unknown_reference` (statut 200).

- 3 **Relecture du statut** : le statut appliqué n'est **pas** celui des paramètres reçus, mais celui que renvoie CamPay quand on l'interroge (`campayTransactionStatus()`). Pourquoi ? Une notification signée pourrait être rejouée ; en redemandant le statut à CamPay, on n'enregistre que la vérité du prestataire.

`paymentApplyCampayStatus()` applique ensuite ce statut de façon **idempotente** : un paiement déjà **PAYE** n'est plus jamais modifié, et un montant confirmé différent du montant attendu est enregistré en **ECHOUÉ** pour vérification manuelle. Le webhook et `ajax/campay_status.php` peuvent donc mettre à jour le même paiement sans conflit.

À retenir

- Webhook = appel de serveur à serveur : sécurité par signature JWT, pas par session.
- On ne croit pas le contenu de la notification : on redemande le statut à CamPay.
- Un paiement **PAYE** est définitif.

Questions de révision

1. **Pourquoi le rôle n'est-il pas un champ du formulaire de connexion ?**

Tout ce qui vient du navigateur est modifiable. Le rôle est déduit côté serveur par `getUserRole()`, selon la table (`administrateur`, `client`, `technicien`, `garage`) où figure l'identifiant.

2. **Pourquoi appeler `session_regenerate_id(true)` après une connexion réussie ?**

Contre la fixation de session : l'identifiant connu avant la connexion est remplacé, un attaquant ne peut donc pas le réutiliser.

3. **Pourquoi les compteurs d'échecs de connexion sont-ils dans des fichiers plutôt qu'en session ?**

Un attaquant peut ne pas renvoyer son cookie et obtenir une session neuve à chaque essai. Les fichiers, indexés par email et par IP, ne dépendent pas de lui.

4. **Que se passe-t-il si une erreur survient au milieu de l'inscription ?**

Tout est dans une transaction : `rollback()` annule toutes les insertions, aucun compte incomplet ne reste en base.

5. **Pourquoi le webhook CamPay relit-il le statut auprès de CamPay au lieu d'utiliser celui reçu ?**

Une notification signée pourrait être rejouée ; seul le statut renvoyé par CamPay pour cette référence fait foi, et un paiement **PAYE** n'est plus modifié.

4. Les quatre espaces : admin/, client/, garage/, technicien/

SmartAutoTrack a quatre types d'utilisateurs : l'**administrateur**, le **client** (particulier ou entreprise), le **garage** partenaire et le **technicien**. Chacun a son propre dossier à la racine du projet. On parle d'« espace » : c'est l'ensemble des pages qu'un rôle a le droit d'ouvrir.

Pourquoi un dossier par rôle ? On voit tout de suite quelles pages sont destinées à qui. Surtout, chaque page ne vérifie qu'une chose au début : le rôle de la personne connectée. Cela limite le risque qu'un client ouvre par erreur une page de garage.

4.1 La structure commune aux quatre espaces

```
admin/ | client/ | garage/ | technicien/
├── dashboard.php      page d'accueil du rôle
├── ... autres pages .php une page = un écran du menu
├── includes/
│   ├── helpers.php    fonctions propres au rôle
│   └── sidebar.php    menu latéral du rôle
├── assets/css/
│   ├── admin_v2.css   habillage de l'espace admin
│   ├── client_v2.css  habillage de l'espace client
│   ├── garage_v2.css  habillage de l'espace garage
│   └── technicien_v2.css habillage de l'espace technicien
```

Le squelette d'une page

Toutes les pages suivent le même ordre. Exemple de `garage/demandes.php` :

- 1 **Chargement de la configuration** : `require_once` `../config/config.php`, `../config/database.php`, `../config/roles.php`, puis `includes/helpers.php` du rôle.
- 2 **Contrôle d'accès** : `requireRole('garage')` (défini dans `config/config.php`). Il faut d'abord être connecté (`requireAuth()`). Une personne connectée avec un autre rôle est renvoyée vers `dashboard.php`, qui la dirige vers son propre espace.
- 3 **Traitement des formulaires (POST)** : le jeton CSRF est vérifié (`verifyCSRFToken()`), les saisies sont validées, puis on écrit en base. Ensuite on redirige (`header('Location: ...?success=...')`). Ainsi, recharger la page ne renvoie pas le formulaire une deuxième fois.
- 4 **Lecture des données** à afficher (requêtes préparées PDO).
- 5 **Affichage** : on déclare la feuille CSS du rôle (`$extraStylesheets = ['assets/css/garage_v2.css']`), puis `include` `../includes/header.php`. On règle les variables de la sidebar (`$activeNav`, `badge`) et on fait `include` `includes/sidebar.php`. Viennent ensuite le contenu, puis `include` `../includes/footer.php`.

helpers.php : des fonctions partagées sans copier-coller

- `client/includes/helpers.php` contient des outils d'affichage généraux, par exemple `v2_relative()` et `v2_today_fr()` (la date du jour en français, sans `strftime()`, déprécié depuis PHP 8.1). Il contient aussi `v2_recommend_garages()`. Chaque fonction est déclarée à l'intérieur d'un `if (!function_exists(...))`.
- Les helpers admin, garage et technicien commencent tous par `require_once __DIR__ .`
`../../client/includes/helpers.php`. **Pourquoi ?** Ces fonctions ne dépendent d'aucun rôle. On les réutilise donc au lieu de les copier.
- Chaque rôle a ensuite ses fonctions préfixées : `av2_` pour l'admin (`av2_donut_svg`, `av2_trend_svg`, `av2_status_badge`, `admin_reassign_garage`), `gv2_` et `garage_` pour le garage (`gv2_donut_svg`, `garage_log`, `garage_status_info`), `tv2_` et `technicien_` pour le technicien (`tv2_phrase`, `technicien_log`). Le préfixe évite que deux fonctions portent le même nom.
- `garage_log()` et `technicien_log()` écrivent dans la table commune `journalactivites`, comme `log_activity()` dans `includes/activity_log.php`. Elles remplissent d'office l'acteur, c'est-à-dire l'utilisateur de la session.

sidebar.php : un menu par rôle

Avant d'inclure la sidebar, la page indique l'entrée du menu à surligner (`$activeNav`). Elle peut aussi donner un nombre à afficher en badge : `$garagesEnAttenteBadge` (admin), `$interventionsBadge` (client), `$pendingBadge` (garage) ou `$tachesBadge` (technicien). Tous les liens sont construits avec `SITE_URL`. La même sidebar marche donc depuis `messages/` ou la racine (`profile.php`). Côté client, `$isEntreprise` change les libellés (« parc ») et ajoute l'entrée « Anomalies ».

Les feuilles CSS « v2 » par rôle

`includes/header.php` ajoute les feuilles listées dans `$extraStylesheets`. Chaque espace charge la sienne. Toutes partagent la même base visuelle (polices Sora et Manrope, sidebar avec logo, cartes arrondies), mais chaque rôle a sa couleur : bleu pour le client et l'admin, teal/cyan pour le garage, ambre/cuivre pour le technicien. **Pourquoi ?** Un coup d'œil suffit pour savoir dans quel espace on se trouve. L'assistant IA charge en plus `assets/css/assistant.css`.

À retenir

- Un dossier par rôle. Chaque page commence par `requireRole()`.
- `includes/helpers.php` contient les fonctions du rôle et réutilise celles du client. `includes/sidebar.php` affiche le menu (`$activeNav` + badge).
- Chaque POST suit le même ordre : CSRF, validation, écriture, redirection.

4.2 L'espace administrateur : admin/

L'admin supervise toute la plateforme. Il valide les garages et les techniciens, affecte ou réaffecte les demandes et consulte les statistiques et le journal. La plupart de ses pages sont **en lecture seule**.

Fichier	Rôle	Accès	Tables / dépendances
<code>dashboard.php</code>	Accueil : compteurs, donut des interventions, tendance sur 7 jours, garages à valider, demandes non affectées, charge des garages, dernières activités.	admin, lecture	client, garage, technicien, vehicule, intervention, anomalie, reparation, utilisateur ; <code>av2_donut_svg</code> , <code>av2_trend_svg</code>
<code>clients.php</code>	Liste des clients (filtres <code>search</code> , <code>type</code>). Suppression d'un client (CSRF), dans une transaction.	admin	utilisateur, client, entreprise, vehicule, anomalie ; suppression de paiement, intervention, vehicule, utilisateur ; <code>log_activity()</code>
<code>client_detail.php</code>	Fiche d'un client : véhicules, 10 dernières interventions, anomalies actives.	admin, lecture	utilisateur, client, entreprise, particulier, vehicule, intervention, garage, anomalie
<code>garages.php</code>	Liste des garages. Création (directement VALIDE), validation ou rejet, suspension ou réactivation.	admin, CSRF	garage, utilisateur, technicien, intervention, notifications, journal ; <code>passwordPolicyError()</code>
<code>garage_detail.php</code>	Fiche d'un garage : validation ou rejet des documents déposés. Chaque décision est journalisée et notifiée.	admin, CSRF	garage, utilisateur, documentgarage, technicien, intervention, notifications
<code>techniciens.php</code>	Techniciens : création (interne ou de garage), rattachement à un garage ou détachement, validation ou rejet, suspension ou réactivation.	admin, CSRF	utilisateur, technicien, garage, intervention, notifications, journal
<code>vehicules.php</code>	Parc de tous les clients (filtres, 200 au plus).	admin, lecture	vehicule, utilisateur, client, anomalie, intervention
<code>interventions.php</code>	Toutes les interventions. Affectation à un garage, création assignée à un technicien, changement de statut.	admin, CSRF	intervention, vehicule, utilisateur, technicien, garage, notifications ; <code>admin_reassign_garage()</code>
<code>intervention_detail.php</code>	Fiche d'une intervention avec tout son historique. Réaffectation à un autre garage.	admin, CSRF	intervention, vehicule, utilisateur, garage, journal ; <code>admin_reassign_garage()</code>
<code>reparations.php</code>	Toutes les réparations (filtres, 200 au plus).	admin, lecture	reparation, intervention, vehicule, utilisateur, garage
<code>anomalies.php</code>	Toutes les anomalies (filtres statut, garage, technicien, dates).	admin, lecture	anomalie, vehicule, utilisateur, intervention, garage, technicien
<code>abonnements.php</code>	Abonnements des clients. Une seule formule existe : « Gratuit ».	admin, lecture	utilisateur, client, entreprise, vehicule
<code>transactions.php</code>	Paiements des clients (filtre <code>statut</code> , 200 au plus).	admin, lecture	paiement, utilisateur, intervention ; <code>paymentsReady()</code>

Fichier	Rôle	Accès	Tables / dépendances
<code>statistics.php</code>	Totaux, évolution sur 12 mois, top 5 des techniciens, anomalies par type. Les graphiques sont dessinés dans le navigateur.	admin, lecture	utilisateur, client, technicien, administrateur, vehicule, anomalie, intervention, reparation
<code>journal.php</code>	Journal d'activité de toute la plateforme (filtres, 300 entrées au plus).	admin, lecture	<code>activity_log_fetch()</code> , garage
<code>assistant.php</code>	Interface de l'assistant IA. Les messages partent en AJAX via <code>assets/js/assistant.js</code> . Sans <code>HF_TOKEN</code> , la page affiche une consigne de configuration.	admin	<code>includes/ai.php</code> (<code>aiHistory()</code>)
<code>settings.php</code>	Choix du thème (liste blanche : light, dark, blue, green, purple, orange).	admin, CSRF	utilisateur.themePreference
<code>includes/helpers.php</code>	Badges et libellés de statut, graphiques SVG, <code>admin_reassign_garage()</code> (règle d'affectation commune à deux pages).	inclus	réutilise <code>client/includes/helpers.php</code>
<code>includes/sidebar.php</code>	Menu admin, avec en badge le nombre de garages à valider.	inclus	<code>\$activeNav</code> , <code>\$garagesEnAttenteBadge</code>

Pourquoi `admin_reassign_garage()` est-elle dans helpers ? La même règle sert dans `interventions.php` et dans `intervention_detail.php`. On l'écrit une seule fois pour que les deux pages ne puissent jamais diverger. La réaffectation n'est permise que si l'intervention est `PLANIFIEE` ou `ANNULEE`, et vers un garage `VALIDE`. L'intervention est alors remise en `PLANIFIEE` avec `idTechnicien = NULL`, car l'ancien technicien n'appartient pas forcément au nouveau garage.

4.3 L'espace client : client/

Le client gère ses véhicules et demande des interventions. Il consulte ses anomalies et ses réparations, et paie en Mobile Money.

Cloisonnement : chaque requête est limitée aux données du client connecté (`v.idClient` ou `i.idClient` égal à l'utilisateur de la session).

Fichier	Rôle	Accès	Tables / dépendances
<code>dashboard.php</code>	Accueil : véhicules et leur dernière anomalie active, interventions actives (5 au plus), dernières réparations, messages non lus, notifications.	client, lecture	vehicule, anomalie, intervention, garage, utilisateur, reparation, messages, notifications
<code>vehicles.php</code>	Ajout, modification et suppression de véhicules. Pour une entreprise : recherche, filtres et pagination (20 par page).	client, CSRF (ajout, modification)	vehicule ; <code>validatePlate()</code> , <code>normalizePlate()</code> , <code>validateModel()</code> ; <code>ajax/get_vehicle.php</code>
<code>vehicle_details.php</code>	Fiche d'un véhicule du client. Le détail d'une anomalie ou d'une réparation est chargé en AJAX.	client (ses véhicules seulement)	vehicule, anomalie, reparation, intervention, utilisateur
<code>interventions.php</code>	Liste avec filtres. Nouvelle demande (<code>?form=request_intervention</code>) auprès d'un garage <code>VALIDE</code> , avec un garage recommandé.	client, CSRF	intervention, notifications, journal ; <code>v2_recommend_garages()</code>
<code>anomalies.php</code>	Anomalies de ses véhicules (filtres statut, niveau, véhicule). Consultation seule.	client, lecture	anomalie, vehicule, intervention, garage
<code>anomalies_recentes.php</code>	Ancienne page (30 derniers jours, graphiques), à l'ancien gabarit. Elle n'est plus dans le menu.	client, lecture	anomalie, vehicule
<code>reparations.php</code>	Historique des réparations : détail, rapport PDF et paiement CamPay , tous en AJAX.	client, lecture (AJAX pour le reste)	reparation, intervention, vehicule ; <code>includes/payments.php</code> ; <code>ajax/campay_collect.php</code> , <code>ajax/campay_status.php</code>
<code>journal.php</code>	Journal d'activité limité au client (filtres de dates).	client, lecture	<code>activity_log_fetch()</code>
<code>abonnement.php</code>	Page d'information sur l'abonnement.	client, lecture	intervention (badge), vehicule

Fichier	Rôle	Accès	Tables / dépendances
sav.php	Service après-vente : FAQ et lien vers la messagerie avec un administrateur.	client, lecture	utilisateur, administrateur ; messages/index.php
assistant.php	Assistant IA du client (AJAX). Signale si l'IA n'est pas configurée.	client	includes/ai.php (aiIsConfigured())
settings.php	Choix du thème (liste blanche).	client, CSRF	utilisateur.themePreference
includes/helpers.php	Helpers réutilisés par les autres rôles (v2_relative , v2_today_fr) et recommandation de garage.	inclus	garage, intervention (charge)
includes/sidebar.php	Menu client (particulier ou entreprise).	inclus	\$activeNav , \$interventionsBadge , \$isEntreprise

La recommandation de garage (v2_recommend_garages()) s'appuie uniquement sur des données réelles. D'abord la charge du garage (ses interventions PLANIFIEE ou EN_COURS), qui donne une idée de sa disponibilité. Ensuite les mots communs entre l'adresse du client et celle du garage, qui donnent une idée de la proximité : il n'y a pas de GPS en base. Le premier garage est présenté comme « recommandé », mais **le client reste libre** d'en choisir un autre.

4.4 L'espace garage : garage/

Le garage reçoit les demandes, les affecte à ses techniciens, suit les interventions et peut renseigner lui-même les réparations et les anomalies. Toutes les requêtes sont limitées à idGarage , le garage connecté (tiré de son profil).

Fichier	Rôle	Accès	Tables / dépendances
dashboard.php	Statistiques, donut par statut, tendance sur 7 jours, demandes non affectées, charge des techniciens, messages non lus.	garage, lecture	intervention, reparation, technicien, utilisateur, messages
demandes.php	Demandes reçues. form=assign : choix d'un technicien VALIDE du garage, de la date et de la priorité, puis notification du technicien et du client. form=refuse : statut ANNULEE , puis notification du client.	garage, CSRF	intervention, technicien, vehicule, utilisateur, notifications ; garage_log()
interventions.php	Toutes les interventions du garage. form=start : une intervention PLANIFIEE déjà affectée passe EN_COURS .	garage, CSRF	intervention, vehicule, utilisateur, technicien
reparations.php	Saisie de la réparation d'une intervention EN_COURS (transaction : réparation, clôture, anomalies résolues, notification).	garage, CSRF	reparation, intervention, anomalie, notifications
anomalies.php	Constater une anomalie sur une intervention EN_COURS ou TERMINEE du garage (statut NOUVELLE). Filtres.	garage, CSRF	anomalie, intervention, vehicule, utilisateur
techniciens.php	Équipe : création d'un technicien rattaché au garage (directement VALIDE), suspension ou réactivation.	garage, CSRF	utilisateur, technicien, notifications ; passwordPolicyError()
vehicule.php	Fiche d'un véhicule, seulement s'il a au moins une intervention dans ce garage.	garage, lecture	vehicule, utilisateur, intervention, reparation, anomalie
historique.php	Interventions terminées ou annulées, réparations et anomalies (100 au plus par liste, filtre de dates).	garage, lecture	intervention, reparation, anomalie, vehicule, utilisateur
journal.php	Journal du garage (catégorie, dates).	garage, lecture	activity_log_fetch()
parametres.php	Thème, dépôt de documents (PDF, JPG ou PNG, 5 Mo au plus, statut EN_ATTENTE) et retrait d'un document encore en attente.	garage, CSRF	utilisateur, documentgarage ; uploads/garages/ , ajax/download_garage_document.php

Fichier	Rôle	Accès	Tables / dépendances
<code>includes/helpers.php</code>	<code>garage_log()</code> , <code>garage_status_info()</code> (libellés « Nouvelle demande », « Planifiée », « En cours », « Terminée », « Refusée / Annulée »), graphiques SVG.	inclus	journalactivites ; réutilise les helpers client
<code>includes/sidebar.php</code>	Menu garage, avec en badge le nombre de demandes non affectées.	inclus	<code>\$activeNav</code> , <code>\$pendingBadge</code> , <code>\$garageNom</code>

Des statuts calculés, pas stockés : une « Nouvelle demande » est une intervention `PLANIFIEE` sans technicien (`idTechnicien IS NULL`). Une fois le technicien affecté, elle est affichée « Planifiée » (`garage_status_info()`). On n'a pas ajouté de valeur au type SQL : la combinaison `statut + idTechnicien` suffit.

4.5 L'espace technicien : technicien/

Le technicien ne voit que **ses** interventions : toutes les requêtes sont limitées à `idTechnicien`, l'utilisateur connecté. Tant que son compte n'est pas validé, le tableau de bord n'affiche qu'un message d'attente.

Fichier	Rôle	Accès	Tables / dépendances
<code>dashboard.php</code>	Statistiques, donut, tendance sur 7 jours, tâches à traiter, aperçu du journal, messages non lus.	technicien, lecture	intervention, reparation, anomalie, vehicule, messages ; <code>activity_log_fetch()</code>
<code>taches.php</code>	Vue courte : interventions à démarrer ou en cours. <code>form=start</code> : passage <code>PLANIFIEE</code> → <code>EN_COURS</code> .	technicien, CSRF	intervention, vehicule, utilisateur ; <code>technicien_log()</code>
<code>interventions.php</code>	Toutes mes interventions (tous statuts, filtres). Même action « démarrer ».	technicien, CSRF	intervention, vehicule, utilisateur
<code>reparations.php</code>	Saisie de la réparation d'une de mes interventions <code>EN_COURS</code> (même transaction que côté garage).	technicien, CSRF	reparation, intervention, anomalie, notifications
<code>anomalies.php</code>	Constater une anomalie sur une de mes interventions <code>EN_COURS</code> ou <code>TERMINEE</code> (statut <code>NOUVELLE</code>).	technicien, CSRF	anomalie, intervention, vehicule
<code>historique.php</code>	Interventions terminées ou annulées, réparations et anomalies (100 au plus par liste).	technicien, lecture	intervention, reparation, anomalie, vehicule, utilisateur
<code>journal.php</code>	Mon journal d'activité (catégorie, dates).	technicien, lecture	<code>activity_log_fetch()</code>
<code>parametres.php</code>	Choix du thème (clair ou sombre).	technicien, CSRF	utilisateur.themePreference
<code>includes/helpers.php</code>	<code>technicien_log()</code> (acteur et technicien renseignés d'office avec l'utilisateur de la session), libellés de statut, <code>tv2_*</code> .	inclus	journalactivites ; réutilise les helpers client
<code>includes/sidebar.php</code>	Menu technicien, avec en badge le nombre de tâches à traiter.	inclus	<code>\$activeNav</code> , <code>\$tachesBadge</code>

À retenir

- L'admin supervise et arbitre (validations, réaffectation). La plupart de ses pages sont en lecture seule.
- Le cloisonnement se fait dans le SQL : `idClient`, `idGarage` ou `idTechnicien` égal à l'utilisateur connecté, dans chaque requête.
- « Nouvelle demande » et « Planifiée » sont calculées à partir de `statut + idTechnicien`. Aucune valeur SQL n'a été ajoutée.

4.6 Parcours métier : le cycle de vie d'une intervention

Voici le fil rouge de l'application. Les quatre espaces y interviennent chacun à leur tour.

- 1 **Demande du client** (`client/interventions.php`) : il choisit un de ses véhicules, un garage VALIDE, un type et une description. La ligne `intervention` est créée avec `idGarage` rempli et sans technicien. Le statut n'est pas précisé dans l'INSERT : c'est la valeur par défaut de la colonne qui s'applique, lue ensuite comme `PLANIFIEE`. L'événement est journalisé, les administrateurs et le garage sont notifiés.
- 2 **Affectation par le garage** (`garage/demandes.php`, `form=assign`) : le garage choisit un technicien. Le code vérifie que ce technicien appartient à ce garage et qu'il est VALIDE. Le garage fixe aussi la date (à 8 h) et la priorité (BASSE, MOYENNE ou HAUTE). Le statut reste `PLANIFIEE` et `idTechnicien` est rempli. Le technicien et le client sont notifiés.
Variante : si le garage refuse (`form=refuse`), le statut passe à `ANNULEE`. L'admin peut alors réaffecter la demande à un autre garage (`admin_reassign_garage()`), ce qui la remet en `PLANIFIEE`.
- 3 **Démarrage** (`technicien/taches.php`, `technicien/interventions.php` OU `garage/interventions.php`, `form=start`) : c'est possible seulement si l'intervention est `PLANIFIEE` et a un technicien. Elle passe alors `EN_COURS`.
- 4 **Anomalies constatées** (`technicien/anomalies.php`, `garage/anomalies.php`) : sur une intervention `EN_COURS` ou `TERMINEE`, on crée une `anomalie` rattachée au véhicule **et** à l'intervention (`idIntervention`), avec le statut `NOUVELLE`. Le client la voit dans `client/anomalies.php`, en consultation seule.
- 5 **Réparation et clôture** (`technicien/reparations.php` OU `garage/reparations.php`, `form=new_reparation`) : il faut une intervention `EN_COURS` qui appartient bien à ce technicien ou à ce garage. Titre, description, diagnostic, travaux et durée sont obligatoires, et le coût ne peut pas être négatif. Ensuite, **dans une seule transaction** :
 - on insère la `reparation` avec le statut `TERMINEE` ;
 - on passe l'intervention en `TERMINEE` ;
 - on résout automatiquement les anomalies de cette intervention : `UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW() WHERE idIntervention = ? AND statut IN ('NOUVELLE', 'EN_COURS')` ;
 - on journalise et on notifie le client (« Rapport de réparation disponible ») ;
 - puis `commit()`.

Pourquoi une transaction ? Ces écritures n'ont de sens qu'ensemble. Une réparation enregistrée alors que l'intervention resterait `EN_COURS`, ou une intervention close avec des anomalies encore ouvertes, laisserait la base incohérente. Avec la transaction, soit tout est enregistré, soit rien ne l'est.

Pourquoi seulement les anomalies de la même intervention ? Une réparation règle ce qui a été constaté pendant cette intervention, pas toutes les anomalies du véhicule. Le lien `anomalie.idIntervention` permet de savoir exactement lesquelles fermer. Conséquence : une anomalie ajoutée *après* la clôture, sur une intervention `TERMINEE`, reste `NOUVELLE`.

Étape	intervention.statut	idTechnicien	Qui agit
Demande	PLANIFIEE	NULL	client
Affectation	PLANIFIEE	renseigné	garage (ou admin)
Refus	ANNULEE	NULL	garage, puis réaffectation par l'admin
Démarrage	EN_COURS	renseigné	technicien ou garage
Réparation	TERMINEE	renseigné	technicien ou garage

4.7 Parcours métier : paiement d'une réparation par CamPay

Le client peut payer en Mobile Money (MTN ou Orange, en XAF) une réparation `TERMINEE` dont le coût est supérieur à 0 (`paymentPayableRepair()`). Tout se passe depuis `client/reparations.php`, en AJAX. La logique se trouve dans `includes/payments.php`, et le dialogue avec l'API dans `includes/campay.php`.

- 1 **Lancement (collect)** : `ajax/campay_collect.php` appelle `paymentStart()`. Le numéro est normalisé au format 2376XXXXXXX (`campayNormalizePhone()`). Dans une transaction, avec un `SELECT ... FOR UPDATE` sur la réparation, on refuse si elle est déjà `PAYE` ou si une tentative `EN_ATTENTE` date de moins de 15 minutes (`PAYMENT_PENDING_LOCK_SECONDS = 900`). Sinon, on crée une ligne `paiement` `EN_ATTENTE`, avec la référence `SAT-<idReparation>-<aléatoire>`.
- 2 **Appel à CamPay** (`campayCollect()`), *hors transaction* parce que le réseau peut être lent. CamPay envoie une demande de validation sur le téléphone du client. Si l'appel échoue, le paiement passe `ECHOUÉ`, ce qui libère la réparation pour une nouvelle tentative.

- 3 **Suivi du statut** : la page interroge `ajax/campay_status.php` toutes les 5 secondes, 36 fois au plus (environ 3 minutes). `paymentRefresh()` demande le statut à CamPay tant que le paiement est `EN_ATTENTE`.
- 4 **Webhook** (`webhooks/campay.php`) : CamPay prévient le serveur à la fin de la transaction. La signature JWT est vérifiée avec `CAMPAY_WEBHOOK_KEY`. Le statut appliqué est **toujours redemandé à l'API** (`campayTransactionStatus()`) : une signature rejouée ne suffit donc pas. En local, le webhook n'est pas joignable depuis Internet, et le suivi de l'étape 3 suffit.
- 5 **Passage à PAYE** (`paymentApplyCampayStatus()`) : l'UPDATE contient `WHERE statut <> 'PAYE'`. Un paiement PAYE est donc définitif. La notification « Paiement reçu » et l'entrée au journal ne partent que si `rowCount() === 1`, c'est-à-dire **une seule fois**, même si le webhook et le suivi arrivent en même temps. Si CamPay confirme un montant différent du montant attendu, le paiement passe `ECHOUÉ` et une vérification manuelle est demandée.

Mode démo plafonné : la démo CamPay (`demo.campay.net`, détectée par `campayIsDemo()`) refuse les montants supérieurs à 25 XAF. `campayChargedAmount()` ne débite alors que `min(coût, CAMPAY_DEMO_MAX_AMOUNT)`. Le coût réel reste celui qui est enregistré dans `paiement.montant` et affiché, et le journal précise le montant réellement débité. En production, on débite toujours le montant réel. Le contrôle du montant de l'étape 5 compare avec ce montant plafonné.

Si les colonnes CamPay de `paiement` n'existent pas encore (`paymentsReady()` renvoie false), le paiement en ligne reste désactivé. Les paiements sont supervisés dans `admin/transactions.php`.

À retenir

- Clôture = une seule transaction : réparation TERMINEE, intervention TERMINEE, anomalies de l'intervention passées en TRAITEE.
- Paiement : `EN_ATTENTE`, puis `PAYE` (définitif) ou `ECHOUÉ`. Le statut vient toujours de l'API CamPay, jamais des paramètres reçus.
- En démo, on débite au plus 25 XAF, mais le montant enregistré reste le coût réel.

Questions de révision

1. **Comment empêche-t-on un client d'ouvrir une page du garage ?**
Chaque page commence par `requireRole('garage')`. Un utilisateur connecté avec un autre rôle est renvoyé vers `dashboard.php`, qui le dirige vers son espace.
2. **Pourquoi les helpers garage, technicien et admin incluent-ils `client/includes/helpers.php` ?**
Pour réutiliser les fonctions d'affichage générales (`v2_relative`, `v2_today_fr`) au lieu de les copier. Les `function_exists` et `require_once` évitent qu'une fonction soit déclarée deux fois.
3. **Qu'est-ce qu'une « Nouvelle demande » pour le garage, en base ?**
Une intervention `PLANIFIEE` de ce garage sans technicien (`idTechnicien IS NULL`). C'est un libellé calculé, pas une valeur stockée.
4. **Que se passe-t-il quand un technicien enregistre une réparation ?**
Dans une transaction, la réparation est créée en `TERMINEE`, l'intervention passe en `TERMINEE`, ses anomalies `NOUVELLE` ou `EN_COURS` passent en `TRAITEE` avec `dateResolution`, puis le client est notifié.
5. **Pourquoi le client ne peut-il pas recevoir deux fois « Paiement reçu » ?**
L'UPDATE vers `PAYE` ne touche que les paiements qui ne sont pas encore `PAYE`. La notification ne part que si cet UPDATE a réellement modifié une ligne (`rowCount() === 1`).

5. assets/, uploads/, scripts/, sql/, tests/ et éléments hérités

Les chapitres précédents décrivaient les pages PHP. Celui-ci couvre ce qui les entoure : les fichiers servis au navigateur (`assets/`), les fichiers envoyés par les utilisateurs (`uploads/`), les outils en ligne de commande (`scripts/`), la documentation de la base (`sql/`), les tests automatisés (`tests/`) et les restes d'anciennes versions du projet, qu'il faut savoir reconnaître pour ne pas les confondre avec l'application actuelle.

```
HCH/
├── assets/
│   ├── css/      style.css, themes.css, home.css, auth_v2.css,
│   │             client_v2.css, garage_v2.css, technicien_v2.css,
│   │             admin_v2.css, assistant.css
│   ├── js/       main.js, themes.js, assistant.js
│   └── img/      logo.png, logo-mark.png
├── uploads/      .htaccess, index.php, techniciens/ (+ garages/ créé au besoin)
├── scripts/      .htaccess, migrate_*.php (7 scripts), set_password.php
├── sql/          SCHEMA_ACTUEL.md, SCHEMA_NOTES.md, init.sql, schema.sql, seed_demo.sql
├── tests/        bootstrap.php, Unit/, Integration/, Functional/
├── Éléments hérités (non utilisés) :
├── api/          ancienne API REST (base smartautotrack)
├── accueil/      admin-dashboard/ "Dashboad Technicien/" "dashbord client/"
├── charles (1).sql, db_init.sql, update_database.sql,
└── update_reparations_table.sql, fix_database_simple.sql
```

5.1 assets/ : ce que reçoit le navigateur

Comment les fichiers sont chargés

Toutes les pages passent par `includes/header.php`, qui charge **toujours** `style.css` et `themes.css`, puis jQuery 3.7.1 (depuis `code.jquery.com`), `main.js` et `themes.js`. Une page peut ajouter ses propres feuilles en remplissant la variable `$extraStylesheets` avant d'inclure l'en-tête, par exemple `$extraStylesheets = [ 'assets/css/garage_v2.css' ]`; . C'est ainsi que chaque espace reçoit son habillage sans dupliquer l'en-tête.

assets/css : une feuille par usage

Fichier	Rôle	Accès	Tables / dépendances
<code>style.css</code>	Styles de base historiques (charte bleu clair <code>#1E90FF</code>) : barre de navigation, boutons, formulaires, toasts, modale, liste des règles de mot de passe <code>.pw-rules</code> .	Toutes les pages	Chargé par <code>includes/header.php</code>
<code>themes.css</code>	Variables CSS des six thèmes : <code>light</code> , <code>dark</code> , <code>blue</code> , <code>green</code> , <code>purple</code> , <code>orange</code> . Le thème actif est choisi par l'attribut <code>data-theme</code> de <code><html></code> .	Toutes les pages	Chargé par <code>includes/header.php</code> ; piloté par <code>themes.js</code>
<code>home.css</code>	Vitrine publique (bleu nuit <code>#0B1330</code> et bleu vif <code>#3956E8</code> , une couleur par rôle).	Public	<code>index.php</code>
<code>auth_v2.css</code>	Carte de connexion centrée avec le logo en grand (classes <code>authv2-</code>).	Public	<code>auth/login.php</code>
<code>client_v2.css</code>	Habillage de l'espace client (préfixe <code>v2-</code>), barre latérale bleue ; variante plus foncée <code>.v2-sidebar-entreprise</code> pour un client entreprise.	Client	<code>client/*.php</code> , et <code>profile.php</code> / <code>messages/index.php</code> quand le rôle est client
<code>garage_v2.css</code>	Habillage de l'espace garage, palette teal <code>#0D9488</code> (préfixes <code>gv2-</code> / <code>g-</code>).	Garage	<code>garage/*.php</code> , <code>profile.php</code> , <code>messages/index.php</code>
<code>technicien_v2.css</code>	Habillage de l'espace technicien, palette ambre <code>#D97706</code> (préfixes <code>tv2-</code> / <code>t-</code>).	Technicien	<code>technicien/*.php</code> , <code>profile.php</code> , <code>messages/index.php</code>
<code>admin_v2.css</code>	Habillage de l'espace administrateur (préfixes <code>av2-</code> / <code>a-</code>), même bleu que le client, sur une barre latérale très sombre <code>#131B3E</code> .	Admin	<code>admin/*.php</code> , <code>profile.php</code> , <code>messages/index.php</code>

Fichier	Rôle	Accès	Tables / dépendances
<code>assistant.css</code>	Bulles de conversation de l'assistant IA (<code>.ai-chat-log</code> , <code>.ai-msg-user</code> , <code>.ai-msg-assistant</code> ...).	Client, Admin	<code>client/assistant.php</code> , <code>admin/assistant.php</code>

Pourquoi un préfixe différent par espace ? Les quatre tableaux de bord partagent le même gabarit (`{préfixe}-shell` , `{préfixe}-sidebar` , `{préfixe}-main`) mais ont chacun leur couleur. Avec des préfixes distincts (`v2` , `gv2` , `tv2` , `av2`), deux feuilles ne peuvent jamais se marcher dessus, et un même script JavaScript peut gérer les quatre en détectant simplement le préfixe présent.

assets/js/main.js : le script commun

Chargé sur toutes les pages. Il regroupe tout ce qui doit fonctionner partout :

- **Jeton CSRF pour l'AJAX.** La fonction `smartautotrackCsrfToken()` lit le jeton publié par `header.php` dans `<meta name="csrf-token">`. Grâce à `jQuery.ajaxSetup`, chaque requête jQuery vers le site reçoit l'en-tête `X-CSRF-Token`, que le serveur vérifie (`config/config.php`). Les requêtes vers un **autre domaine** ne le reçoivent pas, pour ne jamais divulguer le jeton.
- **Objet global `window.SmartAutoTrack`** : utilitaires (dates, échappement HTML, copie...), notifications, messages, modale, toasts, formulaires, tableaux. Un seul espace de noms évite de multiplier les fonctions globales.
- **Notifications et messages.** Pour un utilisateur connecté, le script charge `ajax/notifications.php` et `ajax/message_count.php`, puis **les rafraîchit toutes les 30 secondes**. Le marquage « lu » passe par `ajax/mark_notification_read.php` et `ajax/mark_all_notifications_read.php`. Le titre et le message sont échappés avant affichage, car ils viennent de la base.
- **Erreurs AJAX communes** : 401 → message « Session expirée » puis retour à `auth/login.php` ; 403 → « Accès refusé » ; 5xx → message générique (le détail reste dans les journaux du serveur).
- **Menu mobile des tableaux de bord.** Le script cherche lequel des préfixes `v2`, `av2`, `gv2`, `tv2` est présent, puis injecte un bouton « hamburger » et un rideau sombre. En dessous de 720 px, la barre latérale devient un tiroir ; il se ferme au clic sur le rideau, sur un lien, ou avec la touche Échap. Un seul code sert ainsi les quatre espaces sans modifier leurs pages.
- **Règles de mot de passe en direct.** Sous chaque `<input data-password-policy>`, une liste de 5 règles se coche pendant la frappe : 8 caractères minimum, une minuscule, une majuscule, un chiffre, un caractère spécial. Tant qu'une règle manque, `setCustomValidity()` bloque l'envoi du formulaire. Un champ facultatif laissé vide reste valide (cas du profil, quand on ne change pas son mot de passe).
- **Filtres de saisie `data-only`**, appliqués à la frappe et au collage :
 - `digits` : chiffres uniquement ; sur un `type="number"`, les touches `e` `E` `+` `-` (et `.` `,`) sont bloquées ;
 - `letters` : lettres (accents compris), espace, tiret, apostrophe ;
 - `plate` : lettres, chiffres, espace, tiret, **mis en majuscules** ;
 - `model` : lettres, chiffres, espace et `-` `.` `+` `!` `/`.

Le filtre retire les caractères interdits en gardant le curseur à sa place. Il écoute au niveau du `document`, donc il marche aussi sur les champs ajoutés plus tard (modales). Par ailleurs, les champs nommés `nom` / `prenom` sont limités aux lettres et `telephone` aux chiffres, même sans attribut.

À retenir

- Le JavaScript sert au **confort** de l'utilisateur ; c'est le **serveur qui décide**. Chaque règle a son équivalent PHP : `passwordPolicyError()`, `validateDigitsOnly()`, `validateLettersOnly()`, `validatePlate()`, `validateModel()`.
- Les règles de mot de passe existent en deux exemplaires (`main.js` et `includes/password_policy.php`) : si on modifie l'un, il faut modifier l'autre et `tests/Unit/PasswordPolicyTest.php`.

assets/js/themes.js et assistant.js

- `themes.js` définit la classe `ThemeManager` (instance `window.themeManager`). Au chargement, le thème vient d'abord de la **session PHP** (`window.currentTheme`, posé par `header.php`), sinon du `localStorage` (clé `smartautotrack_theme`), sinon `light`. Pourquoi la session d'abord ? Pour que l'utilisateur retrouve son choix sur un autre appareil. Un changement (boutons radio `name="theme"` des paramètres, raccourci Ctrl+T) est enregistré par `fetch` sur `ajax/save_theme.php`, avec l'en-tête `X-CSRF-Token`.
- `assistant.js` est chargé seulement par `client/assistant.php` et `admin/assistant.php` et ne fait rien si le conteneur `#aiChat` est absent. Il envoie les messages à `ajax/assistant_chat.php` (action `reset` pour effacer la conversation). Les

réponses de l'IA sont **d'abord échappées** ; seuls le gras et les titres Markdown sont ensuite convertis en `<strong>`. Ainsi, aucun HTML produit par le modèle n'est exécuté.

assets/img

`logo.png` : logo complet (barre de navigation de `header.php`, `index.php`, pages de connexion et d'inscription). `logo-mark.png` : symbole seul, affiché en haut des quatre barres latérales (`*/includes/sidebar.php`).

5.2 uploads/ : les fichiers envoyés

On y trouve les documents envoyés par les utilisateurs : pièces justificatives d'un technicien à l'inscription dans `uploads/techniciens/`, documents d'un garage dans `uploads/garages/` (créé au premier envoi). Formats acceptés : PDF, JPEG, PNG, 5 Mo au plus. Le type réel est contrôlé avec `finfo` et ne dépend pas de l'extension annoncée.

Double protection :

- `.htaccess` (dans `uploads/` et `uploads/techniciens/`) : Options `-Indexes` et `Require all denied`. Apache refuse donc tout accès direct aux fichiers.
- `index.php` : répond `403 Accès interdit` si quelqu'un ouvre le dossier. C'est un garde-fou si le `.htaccess` n'est pas pris en compte.

Pourquoi ? Ces documents sont personnels. Ils ne sont servis que par `ajax/download_document.php` (table `technician_documents` : un admin ou le technicien propriétaire) et `ajax/download_garage_document.php` (table `documentgarage` : un admin ou le garage propriétaire). Ces scripts vérifient la session et les droits, et répondent `404` en cas de refus. Le contenu de `uploads/techniciens/` n'est pas versionné dans Git (sauf `.htaccess` et `index.php`).

5.3 scripts/ : outils en ligne de commande

Ces scripts modifient la structure de la base ou un compte. Ils ne doivent **jamais** être lancés depuis un navigateur. `scripts/.htaccess` interdit donc tout le dossier au web. En plus, `migrate_structure.php`, `migrate_legacy_data.php`, `migrate_schema.php` et `set_password.php` refusent de s'exécuter hors CLI (`PHP_SAPI !== 'cli'`). Les quatre migrations ponctuelles ne sont protégées que par le `.htaccess`.

Fichier	Rôle	Accès	Tables / dépendances
<code>migrate_structure.php</code>	Complète le schéma actuel avec ce dont le code a besoin : colonnes manquantes, tables <code>messages</code> , <code>notifications</code> , <code>technician_documents</code> , table <code>paiement</code> et ses colonnes <code>CamPay</code> , <code>index</code> . Ne touche à aucune donnée.	CLI, simulation / <code>--apply</code>	<code>utilisateur</code> , <code>technicien</code> , <code>vehicule</code> , <code>anomalie</code> , <code>intervention</code> , <code>reparation</code> , <code>paiement</code> ...
<code>migrate_garage_login.php</code>	Ajoute <code>garage.idUtilisateur</code> (nullable, UNIQUE) : un garage peut enfin se connecter avec son propre compte.	CLI, écrit directement	<code>garage</code> , <code>utilisateur</code>
<code>migrate_anomalie_intervention.php</code>	Ajoute <code>anomalie.idIntervention</code> (clé étrangère, ON DELETE SET NULL). Avant, le lien anomalie → intervention était deviné par la date, ce qui donnait un résultat faux quand deux interventions étaient proches. Cette colonne permet aussi la résolution automatique des anomalies à la clôture d'une réparation.	CLI, écrit directement	<code>anomalie</code> , <code>intervention</code>
<code>migrate_admin_suspension_v2.php</code>	Ajoute la valeur <code>SUSPENDU</code> aux ENUM <code>garage.statutGarage</code> et <code>technicien.statutValidation</code> . Il suffit : <code>isAccountUsable()</code> n'accepte que <code>VALIDE</code> .	CLI, écrit directement	<code>garage</code> , <code>technicien</code>
<code>migrate_journal_activites_v2.php</code>	Rend le journal multi-rôles : ajoute <code>idUtilisateur</code> (qui a agi), <code>idGarage</code> , <code>idReparation</code> , <code>idAnomalie</code> , <code>categorie</code> , et rend <code>idIntervention</code> facultatif.	CLI, écrit directement	<code>journalactivites</code>

Fichier	Rôle	Accès	Tables / dépendances
migrate_legacy_data.php	Reprend les données de l'ancien dump charles (1).sql (utilisateurs, véhicules, interventions, messages, notifications, documents) dans le nouveau schéma, en gardant les mêmes identifiants. Les données sont recopiées dans le script lui-même . Il vérifie chaque id avant insertion, donc on peut le relancer sans créer de doublons.	CLI, simulation / --apply	Tables des rôles, vehicule, intervention, messages, notifications, technician_documents
migrate_schema.php	OBSOLÈTE, ne plus l'exécuter. Visait l'ancien schéma (users, vehicles ...). Conservé pour l'historique.	CLI	Ancien schéma
set_password.php	Définit le mot de passe d'un compte, ou le verrouille.	CLI uniquement	utilisateur.motDePasse, includes/password_policy.php

Simulation puis --apply

migrate_structure.php et migrate_legacy_data.php fonctionnent en deux temps. Lancés sans option, ils **simulent** : ils affichent [à faire] pour chaque action, sans rien écrire. Avec --apply, ils exécutent et affichent [fait]. Pourquoi ? Pour voir ce qui va changer **avant** de toucher à une base réelle. Tous sont **idempotents** : chaque étape vérifie d'abord si la colonne, la table ou la valeur existe déjà. On peut donc les relancer sans risque. Attention : les quatre migrations ponctuelles n'ont pas de mode simulation.

- 1
- php scripts/migrate_structure.php (simulation), puis la même commande avec --apply
- 2
- migrate_garage_login.php, migrate_anomalie_intervention.php, migrate_admin_suspension_v2.php, migrate_journal_activites_v2.php : chacun répond « rien à faire » si la base est déjà à jour
- 3
- Facultatif : php scripts/migrate_legacy_data.php, puis avec --apply

set_password.php

- php scripts/set_password.php <email> : génère un mot de passe aléatoire fort, conforme aux règles, et l'affiche une seule fois ;
- php scripts/set_password.php <email> <motdepasse> : impose un mot de passe. En CLI, il faut **au moins 12 caractères**, en plus des règles communes ;
- php scripts/set_password.php <email> --lock : écrit !COMPTE_VERROUILLE, qui n'est pas un hash valide. password_verify() échoue alors toujours et aucune connexion n'est possible.

C'est le seul moyen de donner un mot de passe à un administrateur : aucune page ne crée de compte admin.

5.4 sql/ et la base charles

Point essentiel : **aucun fichier .sql du projet ne crée la base actuelle**. Le schéma actuel a été créé directement en base le 23/09/2026. La référence est sql/SCHEMA_ACTUEL.md, complétée par scripts/migrate_structure.php.

Fichier	Contenu	Statut
SCHEMA_ACTUEL.md	Tables et colonnes du schéma actuel, colonnes ajoutées par migration, traduction des statuts, contraintes RESTRICT, limites connues	À jour, référence
SCHEMA_NOTES.md	Analyse de l'ancien schéma	Obsolète
schema.sql, seed_demo.sql	Ancien schéma users / vehicles ... (snake_case) et ses données de démonstration	Obsolètes, en-tête « OBSOLÈTE »
init.sql	Schéma de l'ancienne API : base smartautotrack, tables users, vehicles, anomalies, reparations, interventions, push_tokens	Hérité (API api/)

Tables principales du schéma actuel

Table	Rôle
utilisateur	Compte commun : nom, prénom, email, mot de passe haché, téléphone, thème, photo

Table	Rôle
administrateur, client (+ particulier, entreprise), technicien	Une ligne par rôle, même identifiant que l'utilisateur. C'est cette table qui donne le rôle : il n'y a pas de colonne <code>role</code> .
garage, documentgarage	Garage (statut <code>EN_ATTENTE</code> / <code>VALIDE</code> / <code>REJETE</code> / <code>SUSPENDU</code> , compte de connexion <code>idUtilisateur</code>) et ses justificatifs
vehicule	Véhicule d'un client (immatriculation, marque, modèle, kilométrage...)
intervention	Demande de travaux : client, véhicule, garage, technicien, priorité, statut
anomalie	Problème détecté (niveau <code>FAIBLE</code> / <code>MOYEN</code> / <code>CRITIQUE</code> , statut <code>NOUVELLE</code> / <code>EN_COURS</code> / <code>TRAITEE</code> / <code>IGNOREE</code>), liée à une intervention
reparation	Rapport de réparation (diagnostic, travaux, pièces, coût, durée ; statut <code>EN_ATTENTE</code> / <code>EN_COURS</code> / <code>TERMINEE</code>)
paiement	Paiement Mobile Money CamPay (<code>EN_ATTENTE</code> / <code>PAYE</code> / <code>ECHOUÉ</code> , références, opérateur)
journalactivites	Journal des actions importantes, écrit par <code>includes/activity_log.php</code>
messages, notifications, technician_documents	Tables reprises de l'ancien schéma (colonnes en <code>snake_case</code>), rattachées à <code>utilisateur</code> par clé étrangère

Les tables `assistantia` et `analyseia` existent mais ne servent pas : l'historique de l'assistant IA est gardé en session.

À retenir

- Pour installer : importer un export de la structure de `charles`, puis lancer les migrations. **N'importer aucun .sql du dépôt** : ils créent un schéma qui ne correspond plus au code.
- Deux conventions de nommage cohabitent : `camelCase` pour le nouveau schéma (`idutilisateur`), `snake_case` pour les trois tables reprises (`user_id`, `date_envoi`).

5.5 tests/ : la suite PHPUnit

Configuration : `phpunit.xml` déclare trois suites (`Unit`, `Integration`, `Functional`) et la constante `HCH_BASE_URL` (`http://localhost/HCH/`). `tests/bootstrap.php` charge `config/config.php` et `config/database.php`. Composer n'installe que PHPUnit 9.6 et Guzzle 7.9, en `require-dev` : l'application elle-même n'a aucune dépendance Composer.

Fichier	Rôle	Accès	Tables / dépendances
<code>Unit/AiTest.php</code>	Texte décodé et tronqué, limite de débit qui bloque puis se libère, historique borné et séparé par rôle, contexte injecté dans le prompt selon le rôle	Unit	<code>includes/ai.php</code>
<code>Unit/CampayTest.php</code>	Normalisation du téléphone, correspondance des statuts, plafond du montant en démo seulement, signature JWT du webhook (valide, falsifiée, expirée)	Unit	<code>includes/campay.php</code>
<code>Unit/CsrfTest.php</code>	Jeton accepté depuis un champ POST ou l'en-tête, refusé s'il est absent ou faux	Unit	<code>config/config.php</code>
<code>Unit/EscapeTest.php</code>	<code>h()</code> neutralise les attaques XSS ; de plus, parcourt le code des pages pour repérer tout <code><?php echo \$variable ?></code> non échappé	Unit	<code>config/config.php</code> , pages PHP
<code>Unit/LoginThrottleTest.php</code>	Blocage d'un email après trop d'échecs, blocage d'une IP qui essaie beaucoup d'emails, remise à zéro après une connexion réussie (dossier temporaire via <code>HCH_THROTTLE_DIR</code>)	Unit	<code>includes/login_throttle.php</code>

Fichier	Rôle	Accès	Tables / dépendances
Unit/PasswordPolicyTest.php	Mot de passe conforme accepté, refus quand une règle manque, message listant toutes les règles manquantes	Unit	includes/password_policy.php
Unit/UtilTest.php	sanitize(), validation d'email, formatDate()	Unit	config/config.php
Unit/VehicleFieldsTest.php	Modèles réels acceptés (C-HR, ID.4...), immatriculations, normalizePlate() (majuscules, espaces réduits)	Unit	config/config.php
Integration/DatabaseConnectionTest.php	La connexion à la base fonctionne	Integration	config/database.php
Integration/RolesTest.php	Détection du rôle, compte inconnu, blocage d'un technicien non validé, recherche par email. Insère puis supprime des utilisateurs temporaires	Integration	config/roles.php, tables des rôles
Integration/SchemaTest.php	Les 15 tables attendues et les colonnes ajoutées par migration existent ; aucune clé étrangère ne pointe encore vers l'ancienne table users	Integration	information_schema
Functional/HomepageTest.php	index.php répond 200 et contient « SmartAutoTrack »	Functional	Guzzle, Apache local

Les tests d'intégration et fonctionnels sont **ignorés** (`markTestSkipped`) si la base ou Apache ne répondent pas. Ainsi, la suite ne casse pas sur une machine sans serveur. Les tests d'intégration visent la vraie base configurée : à lancer sur une base de développement, jamais en production.

```
composer test          toutes les suites
composer test:unit     tests/Unit (aucune dépendance externe)
composer test:integration tests/Integration (base de données)
composer test:functional tests/Functional (serveur HTTP local)
```

```
Sans Composer global :
php composer.phar test
php vendor/bin/phpunit -c phpunit.xml --testsuite Unit
```

La suite `Unit` compte **76 tests (135 assertions)**, tous réussis. Ce chiffre dépasse le nombre de méthodes, car plusieurs tests utilisent un `@dataProvider` : ils s'exécutent une fois par jeu de données.

5.6 Éléments hérités : pourquoi ils ne sont plus utilisés

Le projet a d'abord été une maquette HTML statique, puis une API REST sur une autre base, avant de devenir l'application PHP actuelle, par rôle. Les traces de ces étapes sont restées dans le dépôt :

Élément	Ce que c'est	Pourquoi il n'est plus utilisé
api/ (index.php, config.php, obd_sim.php, .htaccess)	Ancienne API REST JSON : routes <code>auth</code> , <code>vehicles</code> , <code>anomalies</code> , <code>reparations</code> , <code>interventions</code> , jetons push FCM, et un simulateur OBD-II qui crée des anomalies selon des seuils (batterie < 30 %, freins < 25 %, moteur > 110 °C)	Elle se connecte à une autre base , <code>smartautotrack</code> (variables <code>SAT_DB_*</code>), et son <code>.htaccess</code> vise <code>/smartautotrack/api/</code> . Aucune page ni aucun script du site ne l'appelle. Elle reste joignable depuis le web : il faut la bloquer ou la supprimer avant une mise en ligne.
accueil/, admin-dashboard/, Dashbord Technicien/, dashbord client/	Maquettes statiques (HTML, CSS, petits scripts d'animation), sans PHP ni base de données	Elles ont été remplacées par <code>index.php</code> , <code>auth/</code> et les espaces <code>admin/</code> , <code>technicien/</code> , <code>client/</code> , qui lisent les vraies données. Aucun lien du site n'y mène.
db_init.sql, update_database.sql, update_reparations_table.sql, fix_database_simple.sql (racine)	Création et correctifs de l'ancien schéma (<code>users</code> , <code>reparations</code> ...)	Ils portent l'en-tête « OBSOLETE, NE PLUS EXÉCUTER » : le schéma qu'ils visent n'existe plus.

Élément	Ce que c'est	Pourquoi il n'est plus utilisé
charles (1).sql (racine)	Dump phpMyAdmin de l'ancien schéma du 21/09/2026, avec des données	Il ne sert que de source historique : ses données ont été recopiées dans <code>scripts/migrate_legacy_data.php</code> .

Ces fichiers ne sont pas dangereux tant qu'on ne les exécute pas. Par ailleurs, le `.htaccess` racine refuse de servir au web les `.sql`, `.md`, `.lock`, `.phar`, `composer.json`, `phpunit.xml` et `tmp_*.txt`. Les fichiers `tmp_*.txt` sont des cookies de tests manuels, ignorés par Git.

À retenir

- Code vivant : `admin/`, `client/`, `garage/`, `technicien/`, `auth/`, `ajax/`, `config/`, `includes/`, `assets/`, base `charles`.
- Héritage : `api/` (base `smartautotrack`), les maquettes HTML et tous les `.sql`. On les garde pour l'historique, on ne les exécute pas.

Questions de révision

1. Comment le jeton CSRF arrive-t-il dans les requêtes AJAX ?

`header.php` le publie dans `<meta name="csrf-token">`. `main.js` le lit (`smartautotrackCsrfToken()`) et l'ajoute en en-tête `X-CSRF-Token` à chaque requête jQuery vers le site, via `jQuery.ajaxSetup`. Le serveur le vérifie dans `config/config.php`.

2. Si le JavaScript bloque déjà les mauvaises saisies, pourquoi valider aussi en PHP ?

Le JavaScript peut être désactivé ou contourné (requête envoyée à la main). Seule la validation serveur (`passwordPolicyError()`, `validatePlate()` ...) fait foi ; le JavaScript n'apporte que du confort.

3. Comment un document envoyé par un technicien est-il protégé ?

`uploads/.htaccess` interdit l'accès direct et `index.php` renvoie 403. Le fichier n'est servi que par `ajax/download_document.php`, qui vérifie que l'utilisateur est un admin ou le technicien propriétaire.

4. À quoi sert le mode simulation des scripts de migration ?

Sans `--apply`, le script liste ce qu'il ferait sans rien écrire. On vérifie donc avant de modifier une base réelle. Les scripts sont idempotents : on peut les relancer sans risque.

5. Pourquoi ne pas utiliser `sql/schema.sql` pour installer la base ?

Il décrit l'ancien schéma (`users`, `vehicles` ...), remplacé le 23/09/2026. Le code utilise le nouveau schéma (`utilisateur`, `vehicule` ...), documenté dans `sql/SCHEMA_ACTUEL.md` et complété par `migrate_structure.php`.